

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY



Experimental Analysis of SVM, Logistic Regression, Random Forest, and MLP on Fashion-MNIST

Lab 03: Image Classification Model

Course: CSC14005 - Introduction to Machine Learning

23KHDL1

Students:

Nguyễn Đỗ Bảo – 23127026

Trần Cẩm Huy – 23127056

Vũ Hoàng Minh – 23127427

Nguyễn Thanh Owen – 23127447

Supervisors:

Mr. Bùi Tiến Lên

Mr. Võ Nhật Tân

Mr. Lê Nhật Nam

Ho Chi Minh City, May 11, 2026

TABLE OF CONTENTS

TABLE OF CONTENTS	i
1 Dataset Description	1
1.1 Data Source & Overview	1
1.2 Dataset Size & Format	1
1.3 Meaning of Each Class	1
1.4 Example Images from Different Categories	3
1.5 Class Distribution	4
2 Preprocessing and Exploratory Data Analysis	5
2.1 Data Ingestion, Cleansing, and Serialization	5
2.2 Binary Parsing and Vectorization	5
2.3 Quality Assurance and Cleansing	5
2.4 Data Serialization and Metadata Export	6
2.5 Dimensionality Reduction and Manifold Analysis	6
2.5.1 Principal Component Analysis (PCA)	6
2.5.2 t-Distributed Stochastic Neighbor Embedding (t-SNE)	7
2.5.3 Uniform Manifold Approximation and Projection (UMAP)	8
2.5.4 Minimum Distortion Embedding (PyMDE)	9
3 Classification Models	11
3.1 Overview of Model Selection	11
3.2 Model 1: Support Vector Machine (SVM)	11
3.2.1 Core Idea	11

3.2.2	Implementation Details	12
3.3	Model 2: Logistic Regression (Linear Baseline)	16
3.3.1	Core Idea	16
3.3.2	Implementation Details	16
3.4	Model 3: Random Forest	17
3.4.1	Core Idea	17
3.4.2	Implementation Details	19
3.5	Model 4: Multi-layer Perceptron (MLP)	22
3.5.1	Core Idea	22
3.5.2	Implementation Details	23
4	Experimental Setup	26
4.1	Data Split Strategy	26
4.2	Evaluation Protocol and Metrics	27
4.3	Hyperparameter Selection Method	28
4.4	Technical Environment	30
4.4.1	Software and Libraries	30
4.4.2	Hardware and Runtime Environment	30
4.4.3	Reproducibility and Random Seed	31
5	Analysis Results	32
5.1	Qualitative & Quantitative Analysis	32
5.1.1	Support Vector Maching (SVM)	32
5.1.2	Logistic Regression	38
5.1.3	Random Forest	41
5.1.4	Multi-layer Perceptron (MLP)	52
5.2	Comparative Discussion: Strengths and Weaknesses	56
5.2.1	Support Vector Machine (SVM)	56

5.2.2	Logistic Regression	57
5.2.3	Random Forest	58
5.2.4	Multilayer Perceptron (MLP)	60
5.3	Selection of the Best-Performing Model	61
6	Ablation Study	64
6.1	Baseline Selection	64
6.2	Ablation Experiments	64
6.2.1	Quantitative Results	65
6.3	Critical Analysis	66
6.3.1	Primary Contributor	66
6.3.2	Qualitative Evidence	66
6.3.3	Robustness and Insights	66
7	Conclusions and Insights	67
	Workload Distribution	69
	BIBLIOGRAPHY	70

CHAPTER 1

Dataset Description

1.1. Data Source & Overview

The **Fashion-MNIST** (Xiao et al., 2017) dataset, provided by Zalando Research, serves as a direct drop-in replacement for the original MNIST dataset of handwritten digits. It was designed to provide a more challenging and realistic computer vision task while maintaining the exact same image formatting and dataset structure.

1.2. Dataset Size & Format

The raw dataset is provided in the original **idx-ubyte** binary format, consisting of a total of **70,000 grayscale images** partitioned as follows:

- **Training Set:** 60,000 image/label pairs (**train-images-idx3-ubyte**).
- **Testing Set:** 10,000 image/label pairs (**t10k-images-idx3-ubyte**).
- **Binary Organization:** In the image files, data is stored as a continuous stream of unsigned bytes following a **16-byte metadata header**. Each image consists of $28 \times 28 = 784$ consecutive bytes, where each byte represents a pixel intensity from 0 (black background) to 255 (white/brightest pixel).
- **Label Format:** Labels are stored in separate files following an **8-byte header**, with each label represented by a single byte ranging from 0 to 9.

1.3. Meaning of Each Class

The dataset categorizes images into 10 distinct classes of apparel and accessories. The integer-to-string mapping is defined in Table 1.1.

Label ID	Class Name
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

Table 1.1: Fashion-MNIST Class Mapping

1.4. Example Images from Different Categories

Fashion-MNIST: 10 Random Samples per Category



Figure 1.1

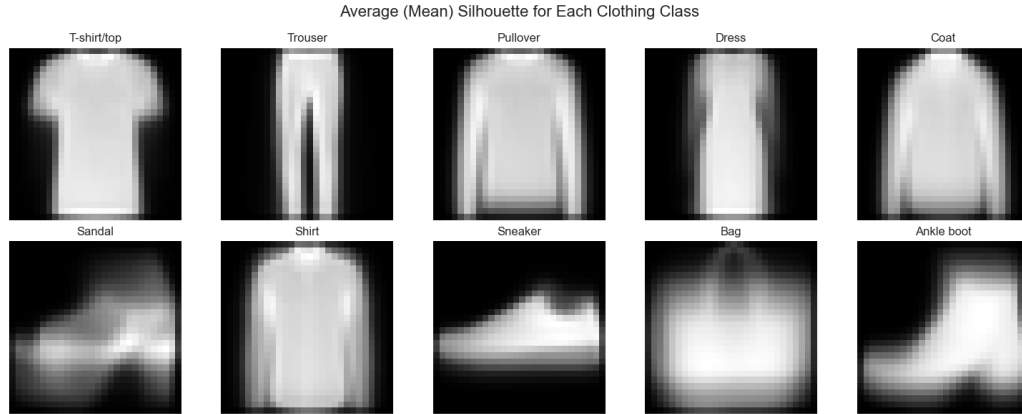


Figure 1.2

1.5. Class Distribution

The Fashion-MNIST dataset is **perfectly balanced**.

- In the training set, each of the 10 classes contains exactly **6,000 images** (10.0% of the dataset).
- In the testing set, each class contains exactly **1,000 images** (10.0% of the dataset).

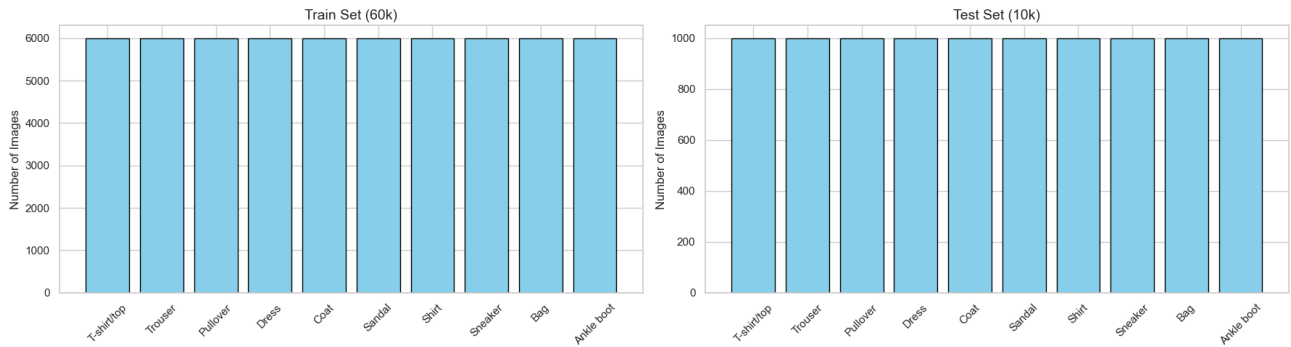


Figure 1.3

CHAPTER 2

Preprocessing and Exploratory Data Analysis

2.1. Data Ingestion, Cleansing, and Serialization

To establish a reliable foundation for our machine learning pipeline, we implemented a robust data processing strategy. This section details the steps taken to ingest the raw binary files, verify their structural integrity, and export the cleaned data for future reproducibility.

2.2. Binary Parsing and Vectorization

The original Fashion-MNIST dataset is distributed in a non-standard binary format (`.idx-ubyte`).

- **Implementation:** We utilized custom parsing functions employing `numpy.frombuffer` to read the byte streams, specifying an offset of 16 bytes for image headers and 8 bytes for label headers. The continuous byte streams were successfully parsed and reshaped into flat 784-dimensional vectors, yielding a (60000, 784) training matrix and a (10000, 784) testing matrix.
- **Justification:** Converting binary-encoded data into structured 2D numeric matrices (samples $\times$ features) is a strict mathematical prerequisite. Machine learning algorithms within the `scikit-learn` and `cuML` ecosystems require this flattened vector format to perform matrix multiplications and calculate loss gradients.

2.3. Quality Assurance and Cleansing

Before executing any statistical profiling or feature engineering, a programmatic integrity check was conducted across the entire feature space to detect anomalies.

- **Implementation:** We systematically scanned both the training and testing matrices for missing values (NaNs), duplicated rows (utilizing `pandas.DataFrame.duplicated`), and completely

blank images (all-zero rows). Additionally, we verified the global minimum and maximum constraints. The results confirmed exceptional data purity: 0 missing values, 0 duplicates, and all pixels strictly confined to the expected $[0, 255]$ integer boundary.

- **Justification:** Ensuring absolute data integrity is critical. Corrupted data, null values, or out-of-bounds pixels can cause fatal execution errors or introduce severe bias into the model’s weights. Confirming the absence of duplicates also guarantees that the validation metrics will not be artificially inflated due to data leakage.

2.4. Data Serialization and Metadata Export

Following the validation of the dataset, the matrices and their corresponding contextual metadata were exported to standardized formats.

The target labels were horizontally prepended to the 784 pixel features, and the combined matrices were serialized into structured `.csv` files. Furthermore, the explicit integer-to-string class mapping (e.g., $0 \rightarrow \text{T-shirt/top}$) was exported as a dedicated `labels.csv` file.

2.5. Dimensionality Reduction and Manifold Analysis

To thoroughly evaluate the spatial structure and class separability of the Fashion-MNIST dataset, we apply and compare four distinct dimensionality reduction techniques. Each method offers a unique mathematical perspective on how the 784-dimensional pixel space can be compressed into a visually interpretable 2D plane.

2.5.1. Principal Component Analysis (PCA)

PCA is a foundational linear transformation technique. It works by identifying the hyperplanes (Principal Components) that capture the maximum amount of global variance in the dataset and projecting the data onto them.

- **Axis Interpretation:** The axes have strict mathematical meaning. The X-axis (PC1) represents the direction of absolute maximum variance, while the Y-axis (PC2) represents the second most significant variance, orthogonal to PC1.
- **Observation:** As seen in Figure 2.1, PCA fails to cleanly separate the classes, retaining

only 46.79% of the variance. The heavy overlapping, particularly among "top-wear" items, mathematically proves that the underlying relationships between clothing pixels are highly non-linear and cannot be resolved by simple linear combinations.

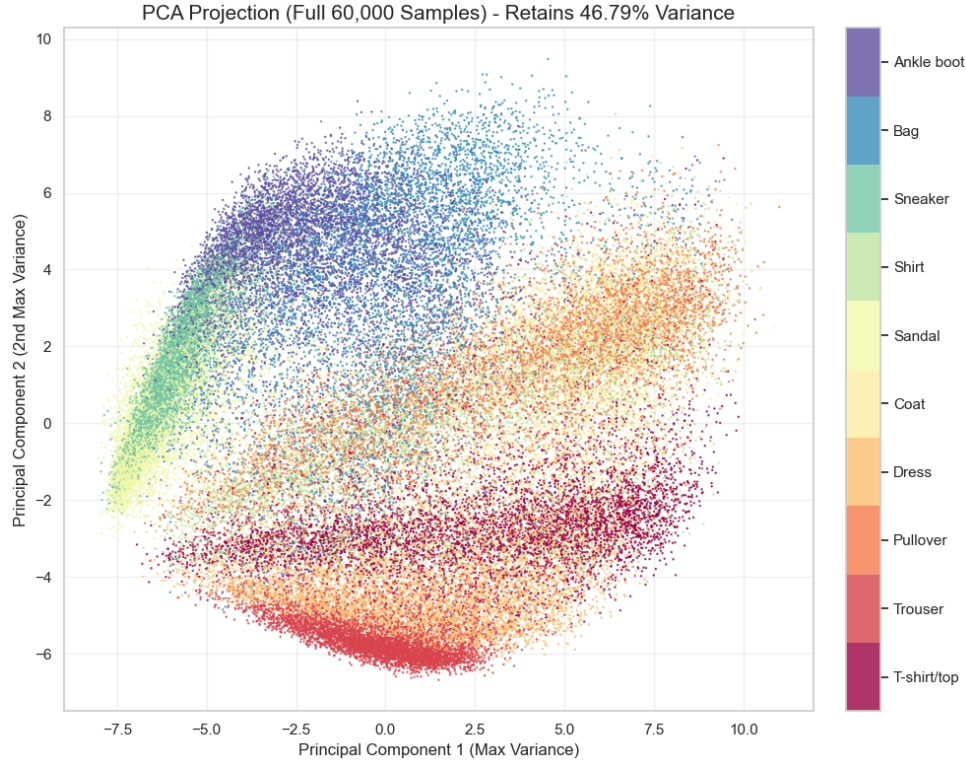


Figure 2.1: PCA Projection: Shows heavy overlapping, indicating linear limitations.

2.5.2. t-Distributed Stochastic Neighbor Embedding (t-SNE)

Unlike PCA, t-SNE is a non-linear, probabilistic algorithm. It computes the probability that two data points are neighbors in the high-dimensional space and meticulously attempts to replicate those probabilities in the 2D embedding. It prioritizes local structure over global distances.

- **Axis Interpretation:** The X and Y axes in t-SNE are arbitrary embedding coordinates. The numerical values on the axes are meaningless; only the relative proximity between points matters.
- **Observation:** Figure 2.2 demonstrates t-SNE's power. It successfully fractures the data into distinct "islands." Footwear classes (Sneaker, Sandal, Ankle boot) are cleanly isolated from

clothing. However, because t-SNE struggles to preserve global topology, the distances between different clusters may not accurately reflect their true morphological differences.

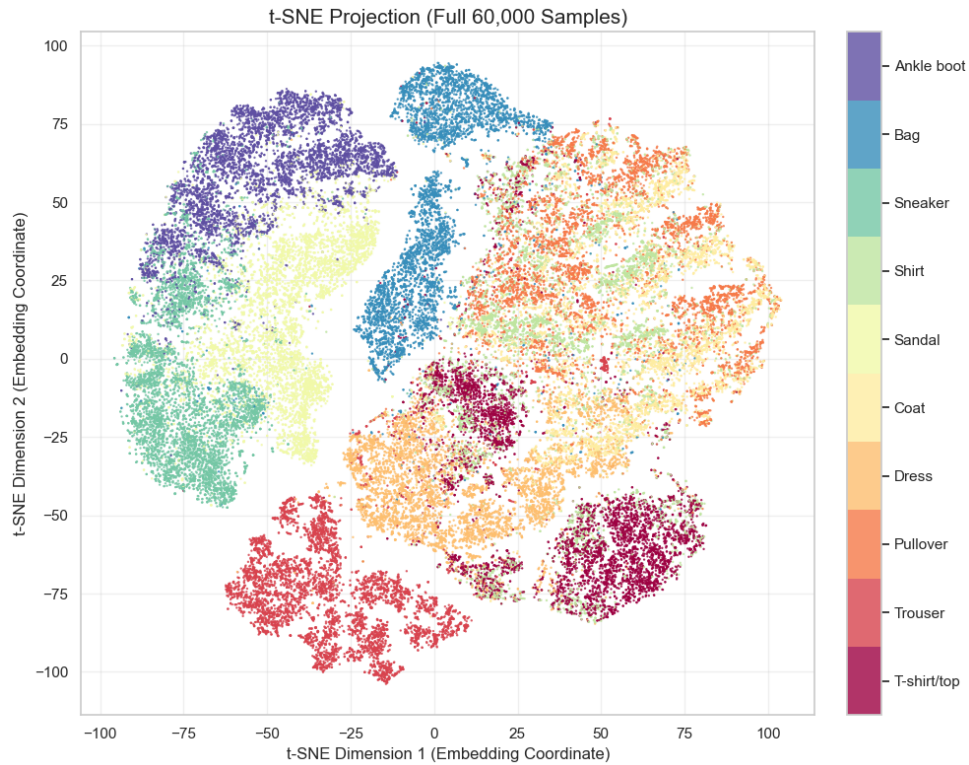


Figure 2.2: t-SNE Projection: Highlights strong local neighborhood preservation.

2.5.3. Uniform Manifold Approximation and Projection (UMAP)

UMAP is a modern manifold learning technique based on Riemannian geometry and algebraic topology. It constructs a high-dimensional topological representation of the data and optimizes a low-dimensional layout to be as structurally similar as possible. It is computationally faster than t-SNE and better at preserving global structure.

- **Axis Interpretation:** Similar to t-SNE, the axes represent an arbitrary topological manifold. However, in UMAP, the empty space (distance) between distinct clusters is globally meaningful, implying a genuine degree of dissimilarity.
- **Observation:** Figure 2.3 provides the clearest holistic view of the dataset. It cleanly separates trousers and footwear while explicitly showcasing the "Upper-Body Bottleneck"—where T-

shirts, Shirts, Coats, and Pullovers merge into a contiguous, overlapping manifold due to their shared torso-and-sleeves silhouette.

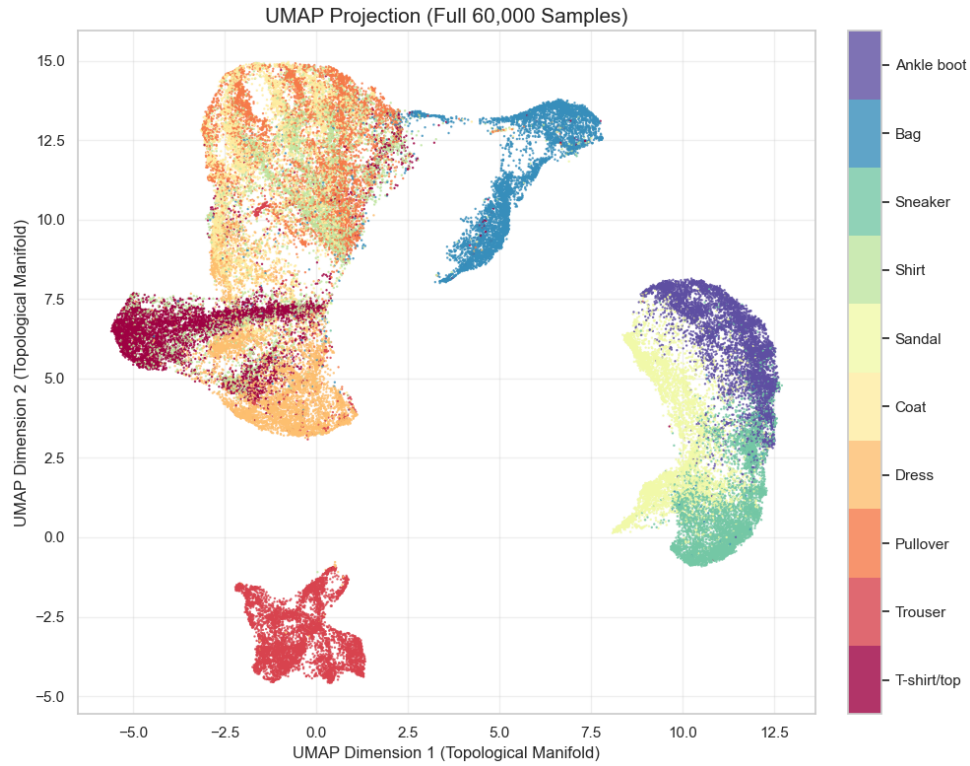


Figure 2.3: UMAP Projection: Balances both local clustering and global topological relationships.

2.5.4. Minimum Distortion Embedding (PyMDE)

PyMDE approaches dimensionality reduction as an optimization problem explicitly designed to minimize the geometric distortion of distances between nearest neighbors.

- **Axis Interpretation:** The axes represent coordinates in a mathematically optimized "Minimal Distortion Space."
- **Observation:** As illustrated in Figure 2.4, PyMDE aggressively compresses similar items together, resulting in extremely dense, tightly packed, and isolated clusters. This visualization emphasizes the most fundamentally distinct geometric shapes in the dataset, further validating the necessity of extracting edge-based features (such as HOG) to differentiate the highly compressed clusters.

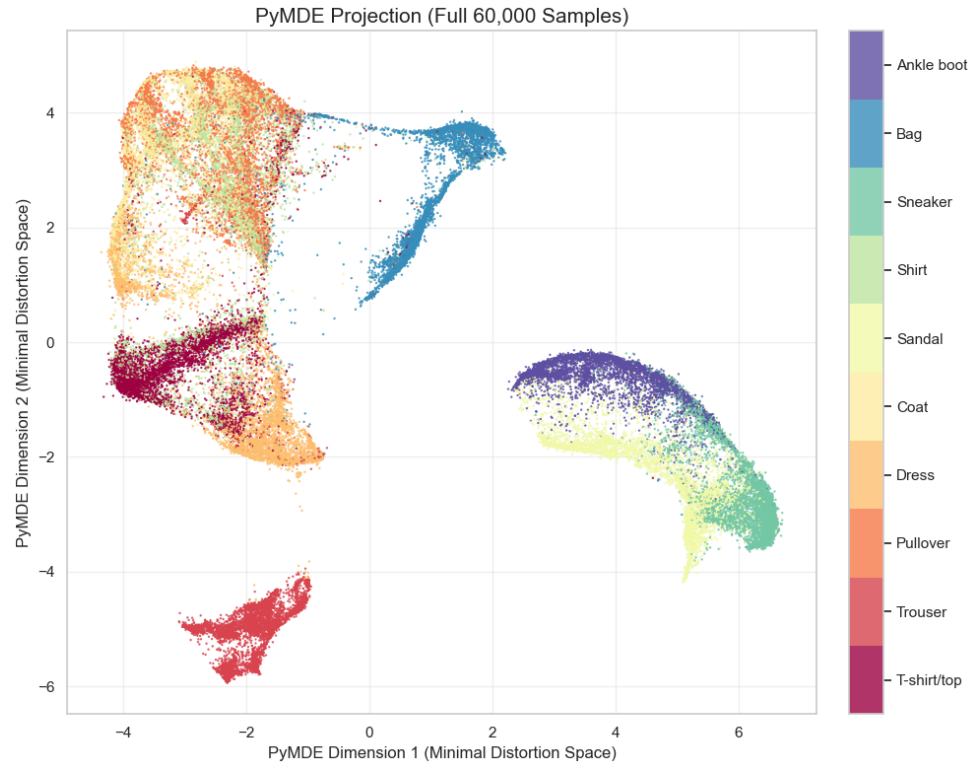


Figure 2.4: PyMDE Projection: Produces highly compact clusters by explicitly minimizing neighbor distance distortion.

CHAPTER 3

Classification Models

This chapter presents the selection and implementation of four classification models. Together, these models cover a broad range of algorithmic families—linear, kernel-based, tree-based ensemble, and neural network methods—allowing for a comprehensive evaluation on the Fashion-MNIST dataset.

3.1. Overview of Model Selection

The study compares four classifiers chosen to represent distinct modeling approaches:

- **Logistic Regression:** A linear model that serves as a simple and interpretable baseline.
- **Support Vector Machine (SVM):** A kernel-based model that can capture more complex decision boundaries.
- **Random Forest:** A tree-based ensemble method that is robust to noise and nonlinearity.
- **Multi-layer Perceptron (MLP):** A neural network model capable of learning richer feature interactions.

3.2. Model 1: Support Vector Machine (SVM)

3.2.1. Core Idea

The Support Vector Machine (SVM) is a robust margin-based classifier designed to construct an optimal decision boundary by maximizing the geometric distance between distinct categories. Unlike end-to-end neural networks that jointly learn feature extraction and classification from scratch, this SVM pipeline relies on a decoupled, mathematically rigorous approach. Raw 784-dimensional grayscale images are first standardized and projected onto a lower-dimensional

manifold using Principal Component Analysis (PCA) to filter noise and mitigate the curse of dimensionality. Because the topological structure of the Fashion-MNIST dataset is highly non-linear, the model employs the Radial Basis Function (RBF) kernel to implicitly map these compressed feature vectors into an infinite-dimensional space, computing the similarity between two samples x and x' as $K(x, x') = \exp(-\gamma\|x - x'\|^2)$. By utilizing this kernel trick alongside rigorous Bayesian optimization to balance the regularization penalty (C) and kernel coefficient (γ), the SVM efficiently resolves complex class boundaries without requiring the extensive computational overhead of deep representation learning.

3.2.2. Implementation Details

Building on the hardware and software environment described in the previous section, the Support Vector Machine (SVM) pipeline was architected for fully GPU-native execution utilizing the NVIDIA RAPIDS cuML library. This design prioritizes computational efficiency by explicitly casting all datasets to 32-bit floating-point tensors and retaining all feature-transformed matrices—from PCA reductions to dense RBF kernel calculations—directly within the GPU VRAM. Consequently, this approach successfully eliminates the severe memory overhead and extended training latencies traditionally associated with CPU-bound SVM optimization on high-dimensional image data.

Baseline and Logic Verification

To establish an absolute minimum performance threshold, a stratified random guessing baseline was first implemented. Given the perfectly balanced 10-class distribution of the Fashion-MNIST dataset, this dummy classifier inherently yielded an expected accuracy and **Macro-F1 score** of approximately **10%**. This theoretical floor provided a critical quantitative reference point to measure the actual representational learning capacity of the subsequent SVM models.

Before jumping into the heavy GPU tuning, we ran a quick "sanity check" to ensure our data pipeline was built correctly.

- **The Initial Test:** We tested if the default SVM could memorize a tiny, isolated batch of 50 training samples. However, it only reached 88% accuracy. This raised a question: was the data corrupted, or were the default settings just too strict?

-
- **The Diagnostic & Conclusion:** To find out, we temporarily removed the model’s regularization penalty by cranking up the parameter to $C = 1000$. The SVM immediately hit a 100% memorization rate. This successfully proved that our features and labels were perfectly matched and free of any numerical bugs, giving us the green light to proceed with full-scale optimization.

Data Pipeline and Preprocessing

Before feeding images into the SVM, we had to clean up and reshape the data. SVMs are incredibly powerful, but they are also quite picky about how the input data is formatted.

- **The 90/10 Split:** First, we needed a fair way to evaluate our model during the tuning phase. We took the 60,000 training images and carved out 10% (6,000 samples) to act as our Validation set. To prevent any class bias, we used a stratified split. This guaranteed that our validation set was perfectly balanced, containing exactly 600 images for each of the 10 clothing categories.
- **Scaling the Pixels:** SVM algorithms figure out class boundaries by calculating the geometric distances between data points. If we feed it raw pixel values ranging from 0 to 255, the math gets messy and the model struggles to converge. To fix this, we ran the data through a **StandardScaler**, shifting all features to have a mean of 0 and a standard deviation of 1.
- **Trimming the Fat (PCA):** Training an RBF-kernel SVM on 784 dimensions (the raw pixels of a 28x28 image) is a computational nightmare and introduces a lot of unnecessary noise. We brought in Principal Component Analysis (PCA) and set it to retain 95% of the cumulative variance. This smart compression shrank our feature space from 784 down to just 256 core dimensions. Essentially, PCA threw away the useless dark background pixels and kept only the vital structural shapes, allowing the model to run significantly faster without losing accuracy.

Bayesian Hyperparameter Fine-Tuning

Now that our data was clean, scaled, and compressed, it was time to dial in the model’s exact settings. Finding the perfect equilibrium for an SVM—specifically the regularization penalty (C)

and the kernel coefficient (γ)—can easily turn into a massive guessing game. Instead of using a slow, brute-force Grid Search, we let the machine do the smart work.

- **The Smart Search (Optuna):** We brought in Optuna and used its **TPESampler** to run **Bayesian Optimization**. Unlike standard random searching, this algorithm actually "learns" from its past attempts, using previous results to make highly educated guesses about where the optimal parameters might be hiding.
- **The Heavy Lifting:** Optuna was given a budget of 20 trials to find the absolute best combination. During each trial, the SVM trained on the 54,000-sample subset and was tested against the 6,000-sample validation set. Our single goal here was to maximize the Macro-F1 score.
- **Understanding Why (fANOVA):** After Optuna confidently locked in the winning combo ($C \approx 17.14$, $\gamma = \text{'scale'}$), we didn't just blindly accept it. We ran an fANOVA analysis to figure out exactly why the model improved. The importance chart made it obvious: C was the primary driving force. Tuning the regularization penalty pushed nearly all the performance gains, while adjusting γ barely moved the needle.

Optimised SVM Configuration

After having magic numbers from Optuna ($C \approx 17.14$ and $\gamma = \text{'scale'}$). But before throwing SVM model at the final test set, we made one crucial, industry-standard tweak to maximize our performance.

- **Going All In (The 60,000 Retrain):** During the tuning phase, we held back 6,000 images for validation. But now that the tuning was completely done, keeping those images locked away was just a waste of good data. In real-world data science, more data almost always equals better decision boundaries.
- **Rebuilding the Pipeline:** We took the entire 60,000-sample training dataset and essentially started fresh. We re-fitted our StandardScaler and PCA on this massive combined dataset to make sure we captured every single ounce of variance available.

-
- **The Final Engine:** Finally, we plugged our optimized parameters into the GPU-accelerated SVM and trained it on the fully preprocessed 60,000 features. By feeding the model 100% of the available historical data, we gave it the absolute best shot at conquering the unseen test images.

Statistical Stability: 5-Seed Verification

A classic trap in machine learning is accidentally tuning a model to perform exceptionally well on just one specific data split. To prove the optimized parameters weren't just a "lucky strike" tied to the initial Seed 42, the pipeline needed a proper stress test.

- **The Testing Setup:** The entire process—from splitting the dataset to initializing the PCA and training the SVM—was executed five separate times using five completely random seeds (42, 5435, 123, 2026, 23).
- **Consistent Results:** If the configuration was fragile, the scores would have fluctuated wildly across different runs. Instead, the performance barely budged. The Macro-F1 score locked in at an average of 0.9040, with a microscopic standard deviation of just ± 0.0050 .
- **The Conclusion:** This minimal variance mathematically guarantees that the model is robust. The high accuracy comes from the SVM successfully learning the underlying image patterns, rather than just getting a favorable train-validation split by chance.

Diagnostic and Interpretability Suite

To look beyond the global accuracy score and understand how the model actually behaves on the test set, three visual tools were used:

- **Optuna Plots (History & Importance):** These charts track the hyperparameter tuning progress to confirm the search converged properly. They also clearly show which parameter (specifically C) had the biggest impact on improving the final score.
- **Confusion Matrix:** This grid maps the actual clothing labels against the predicted ones. It is used to spot specific patterns of mistakes, such as the model frequently confusing visually similar items like "Shirt" and "T-shirt".

-
- **Class-wise F1 Chart:** Instead of just providing a single average score, this chart breaks down the performance for each of the 10 categories. It helps quickly identify the "weak links" in the dataset where the model struggles the most.

3.3. Model 2: Logistic Regression (Linear Baseline)

3.3.1. Core Idea

The selected baseline is a **Multinomial Logistic Regression** classifier optimized with L2 (Ridge) regularization. Unlike binary One-vs-Rest wrappers, this architecture utilizes the Softmax function to jointly model the probability distribution across all 10 mutually exclusive classes, minimizing Categorical Cross-Entropy loss.

The fundamental philosophy behind this model is **geometric abstraction**. Because linear classifiers construct rigid hyperplanes, they inherently fail when applied to raw, overlapping pixel intensities. To overcome this, the core strategy relies on transforming the feature space using the **Histogram of Oriented Gradients (HOG)**. By converting pixel brightness into local edge orientations, the model gains crucial translation and illumination invariance, allowing the linear boundaries to separate garments based on their structural silhouettes rather than raw spatial noise.

3.3.2. Implementation Details

To execute this architecture, a rigorous, GPU-accelerated experimental pipeline was constructed. The implementation is divided into three sequential stages:

1. Feature Engineering and Anti-Leakage Scaling

The raw 784-dimensional pixel space was mathematically compressed into 324 high-density structural features using the HOG algorithm (parameters: 7×7 pixels per cell, 9 orientations).

To guarantee numerical stability for the optimization solver, **Z-score normalization** (`StandardScaler`) was universally applied. To strictly prevent data leakage, the scaler was fitted exclusively on the training subset, ensuring the validation and test distributions remained completely unseen during the transformation phase.

2. Optimization Engine and Hyperparameter Tuning

Model training was executed via the GPU-accelerated `cuML` library, utilizing the Quasi-Newton (`qn`) L-BFGS solver. This second-order optimizer efficiently navigates the loss landscape, providing rapid convergence for large-scale matrices.

For hyperparameter tuning, **Bayesian Optimization** (`Optuna`) was deployed to find the optimal inverse regularization strength (C). The Tree-structured Parzen Estimator (TPE) algorithm searched a logarithmic space to maximize the **Macro-F1 Score** on an isolated 6,000-sample validation set.

3. Verification Protocol

To mathematically ensure that the hyperparameter tuning was not biased by a lucky data split, a **5-Seed Repeated Hold-out Validation** protocol was enforced. For each of the 5 distinct random seeds, the original 60,000-sample dataset was fully re-shuffled, re-split (90% train, 10% validation), and the model was trained from scratch. Only after proving statistical stability across these seeds was the final configuration locked in for testing.

3.4. Model 3: Random Forest

3.4.1. Core Idea

Random Forest is an ensemble learning method that constructs a large collection of decision trees during training and produces a final prediction by aggregating the outputs of the individual trees. In classification tasks, the final prediction is obtained through majority voting, whereas in regression tasks it is obtained by averaging the predictions of all trees. The central premise of the method is that a collection of comparatively weak learners can be combined to form a stronger and more robust predictor with improved generalization to unseen data.

This methodology is based primarily on two mechanisms: **Bootstrap Aggregating (Bagging)** and **Feature Randomness**.

Bootstrap Aggregating (Bagging)

To reduce the high variance commonly associated with individual decision trees, Random Forest employs bagging. Given a training set $\mathcal{D}$ of size N , bagging generates B bootstrap samples $\mathcal{D}_b$, each of size N , by sampling uniformly from $\mathcal{D}$ with replacement. This resampling process is referred to as *bootstrapping*.

A decision tree T_b is then trained on each bootstrap sample $\mathcal{D}_b$. Because sampling is performed with replacement, some observations may appear multiple times in a given bootstrap sample, while others are excluded. The excluded observations are commonly referred to as Out-of-Bag (OOB) samples.

Feature Randomness

In a conventional decision tree, each node is split using the most informative feature among all available predictors. In Random Forest, however, each split is selected from a randomly sampled subset of m predictors drawn from the full set of p predictors, where typically $m \approx \sqrt{p}$ in classification settings. This strategy prevents a small number of dominant predictors from controlling every split and consequently reduces the correlation among trees.

Mathematical Formulation

Let $T_b(x)$ denote the prediction of the b -th decision tree for an input vector x . The ensemble prediction of a Random Forest with B trees is defined as follows:

- **For Regression:** The final prediction is given by the average of the predictions of all trees:

$$\hat{f}_{rf}^B(x) = \frac{1}{B} \sum_{b=1}^B T_b(x) \quad (3.1)$$

- **For Classification:** The final prediction is obtained through majority voting:

$$\hat{C}_{rf}^B(x) = \text{majority vote}\{\hat{C}_b(x)\}_1^B \quad (3.2)$$

Variance Reduction and Generalization

The variance of the average of B identically distributed, though not necessarily independent, random variables with variance σ^2 and pairwise correlation ρ can be expressed as:

$$\text{Var}(\text{Ensemble}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 \quad (3.3)$$

Through the combined use of bootstrap sampling and random feature selection, Random Forest reduces the correlation ρ among trees. As B increases, the term $\frac{1-\rho}{B}\sigma^2$ approaches zero, thereby reducing overall variance without substantially increasing bias. This property explains the strong generalization performance commonly observed in Random Forest models.

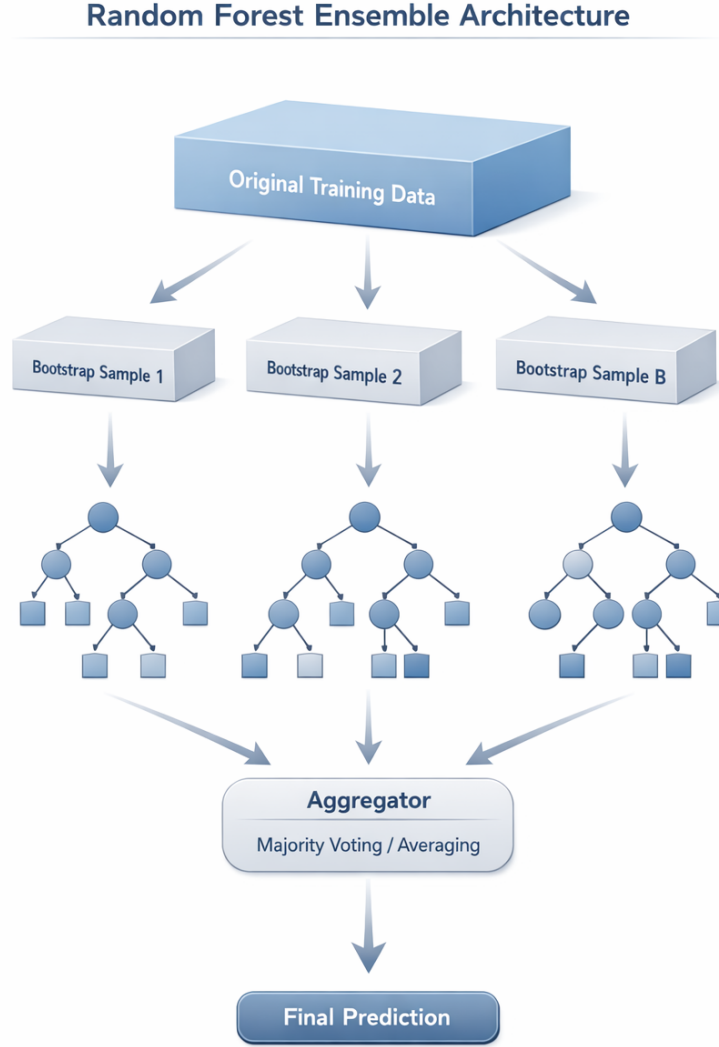


Figure 3.1: Architecture of a Random Forest ensemble.

3.4.2. Implementation Details

Building on the hardware and software environment described in Section 4.4, the Random Forest pipeline was designed for fully GPU-native execution using the NVIDIA RAPIDS suite. This design emphasizes high-throughput data processing by retaining all feature-engineering and

classification tensors in the NVIDIA Tesla T4 VRAM, thereby eliminating the PCIe transfer bottlenecks traditionally associated with CPU-GPU hybrid pipelines.

Baseline and Logic Verification

To establish a performance floor, a baseline was implemented using a `DummyClassifier` with a stratified strategy, yielding an accuracy of approximately 10.1%. Prior to full-scale training, the logic was verified through a “tiny-batch memory test” where the model was required to overfit 50 samples. Achieving 100% accuracy on this subset served as a vital sanity check, confirming the integrity of the gradient path and the transition from HOG features through PCA to the Random Forest classifier. (Müller & Guido, 2016)

Custom GPU-Native Feature Engineering

To resolve the “Upper-Body Bottleneck” identified in Section 2.4, a specialized feature-extraction layer was implemented:

- **PyTorch-HOG Extractor:** Implemented using 2D convolutions (Sobel operators) and `AvgPool2d` in PyTorch, ensuring that HOG computations occur entirely within GPU memory.
- **Descriptor Details:** The extractor utilizes 16 orientation bins and 2×2 pixel cells. This configuration transforms the raw 28×28 input into a 3,136-dimensional feature vector ($14 \times 14 \times 16$), capturing fine-grained structural edges essential for silhouette differentiation.

Dimensionality and Latency Management

To ensure feature scale invariance, standardization was performed via `cuml.StandardScaler`. Subsequently, the high-dimensional feature space $\mathbf{X} \in \mathbb{R}^{N \times 784}$ was transformed using Principal Component Analysis:

- **PCA Projection:** Features were projected onto $k = 110$ principal components using `cuml.PCA`. The selection of k is mathematically justified by the Cumulative Explained Variance Ratio (EVR), $V(k)$, which measures the proportion of total variance captured by the first k eigenvectors:

$$V(k) = \frac{\sum_{i=1}^k \lambda_i}{\text{Tr}(\Sigma)} = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^d \lambda_j} \quad (3.4)$$

where λ_i represents the i -th eigenvalue of the covariance matrix Σ in descending order (Jolliffe, 2002). For this specific implementation, $V(110) \approx 0.29$, indicating that 29% of the global structural variance is preserved. This threshold effectively isolates the primary silhouette signal from high-frequency pixel noise while maintaining a computational complexity of $O(k \log N)$ (Tipping & Bishop, 1999).

- **Inference Latency:** Systematic profiling was conducted following standard GPU deployment protocols. Inference time was averaged over 100 iterations with a batch size of 1. This ensures that the dimensionality reduction to $k = 110$ strikes the optimal balance between information density and the low-latency requirements ($\leq \tau$) necessary for real-time classification.

Optimized Random Forest Configuration

The final model utilizes the `cuml.RandomForestClassifier` with hyperparameters derived via Bayesian optimization (Optuna):

- **Final Parameters:** The optimized forest consists of $B = 1,068$ estimators with a maximum depth of 48. Training convergence was achieved in 21.9 seconds on the Tesla T4.
- **Optimization Heuristics:** The search process employed a `MedianPruner` and an `Early StoppingCallback` with a patience of 10 trials, preventing resource wastage on sub-optimal regions of the parameter space.

Diagnostic and Interpretability Suite

To evaluate model reliability, three specialized diagnostic tools were developed:

1. **Optuna Optimization History and Parameter Importance Plots:** to document search convergence and hyperparameter sensitivity.
2. **Confusion Matrix:** a per-class breakdown of predictions versus true labels to localise systematic errors.
3. **Class-wise F1 Heatmaps:** Visualizing performance across the ten categories to isolate specific class-pair confusion.

-
4. **Learning Curve Profiling:** Accuracy was measured at varying training fractions $f \in \{0.05, 0.1, \dots, 1.0\}$, providing empirical evidence of the model’s sample efficiency and convergence behavior.

3.5. Model 4: Multi-layer Perceptron (MLP)

3.5.1. Core Idea

The Multi-layer Perceptron is a feed-forward artificial neural network composed of an input layer, one or more hidden layers, and an output layer. Unlike the linear or kernel-based models above, an MLP is a *learnable feature extractor*: every hidden layer applies a non-linear transformation whose parameters are jointly optimised with the classifier, so the representation and the decision boundary are trained end-to-end from raw standardised pixels. This property allows the network to capture the non-linear manifold structure of Fashion–MNIST directly, without relying on a fixed hand-crafted descriptor such as HOG.

Network Architecture

The network processes flattened 28×28 grayscale images as 784-dimensional input vectors. Each hidden layer l applies an affine transformation followed by a non-linearity:

$$\mathbf{h}^{(l)} = \phi \left(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right), \quad (3.5)$$

where $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the learned weight matrix and bias vector, $\mathbf{h}^{(0)} = \mathbf{x}$ is the input, and ϕ is the **ReLU** activation $\phi(z) = \max(0, z)$. ReLU is preferred over sigmoid or tanh because it does not saturate for positive inputs, which mitigates the vanishing gradient problem and enables faster convergence of gradient-based optimisers.

The final layer produces class logits $\mathbf{z} \in \mathbb{R}^{10}$, which are converted into a probability distribution via the **Softmax** function:

$$P(y = k \mid \mathbf{x}) = \frac{\exp(z_k)}{\sum_{j=1}^{10} \exp(z_j)}. \quad (3.6)$$

Loss Function and Optimisation

The network minimises the **Categorical Cross-Entropy** loss, which penalises the negative log-likelihood assigned to the true class:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^{10} 1[y_i = k] \log P(y = k \mid \mathbf{x}_i). \quad (3.7)$$

Optimisation is performed with **Adam**, an adaptive first-order optimiser that maintains per-parameter running averages of the first and second moments of the gradient. Adam is chosen because it combines the benefits of momentum with adaptive learning rates, making it robust to sparse gradients and heterogeneous feature scales common in image data.

Regularisation

To counter the over-parameterisation of deep networks on a 54,000-sample training set, we apply L_2 **weight decay** by adding $\alpha \|\theta\|_2^2$ to the loss. This shrinks the magnitude of the learned weights, biases the network toward simpler decision surfaces, and reduces the variance of predictions across random initialisations.

3.5.2. Implementation Details

Building on the hardware and software environment described in Section 4.4, the MLP pipeline was implemented in **PyTorch** with full GPU execution on the NVIDIA Tesla T4. All tensors (inputs, weights, gradients) reside in VRAM throughout training, and mini-batch data transfers are overlapped with computation via the standard **DataLoader** abstraction. The pipeline is fully reproducible through a `seed_everything` utility that fixes the random state of NumPy, PyTorch, and CUDA.

Baseline and Logic Verification

Two sanity checks preceded any hyperparameter search:

- **Performance Floor:** A **DummyClassifier** with a stratified strategy was fitted on the standardised training set, yielding a validation accuracy and macro-F1 of approximately 10.1%. Any real model must substantially exceed this floor.
- **Tiny-Batch Overfit:** A small MLP was trained on a deliberately tiny subset of 50 samples

for 500 full-batch epochs and was required to achieve 100% training accuracy. This confirms that the forward pass, loss computation, backpropagation, and optimiser updates are all wired correctly — if the model cannot memorise 50 points, the training loop is broken.

Data Pipeline and Preprocessing

The original 60,000-sample training set was split in a stratified 90/10 ratio into 54,000 training and 6,000 validation samples using seed 42. Pixel values were standardised with `StandardScaler` (fit on training data only, then applied to validation and test sets) to enforce zero mean and unit variance. This standardisation is essential: without it, the large input magnitudes (0–255) saturate the early layers and destabilise Adam’s adaptive step sizes. Training data was served in mini-batches of size 256 via a shuffled PyTorch `DataLoader`.

Bayesian Hyperparameter Fine-Tuning

Hyperparameters were tuned with the `Optuna` framework using the Tree-structured Parzen Estimator (TPE) sampler. The search space was:

- **Number of hidden layers** $\in \{1, 2, 3\}$.
- **Units per layer** $\in [32, 512]$ (integer, independently per layer).
- **Weight decay** $\alpha \in [10^{-5}, 10^{-1}]$ (log-uniform).
- **Learning rate** $\eta \in [10^{-4}, 10^{-2}]$ (log-uniform).

The objective was to maximise **Macro-F1** on the 6,000-sample validation set, ensuring that performance on difficult minority-behaviour classes (e.g. Shirt) was treated on equal footing with easy classes. Each trial trained for up to 50 epochs with early stopping (patience 5), and the `MedianPruner` terminated underperforming trials after a warm-up of 10 epochs to conserve GPU time. With a fixed budget of **100** trials, the study completed in approximately **22.3** minutes on the Tesla T4.

Optimised MLP Configuration

The search converged on a two-hidden-layer architecture with the following parameters:

- **Hidden layers:** [272, 385] (two layers with 272 and 385 ReLU units respectively).

-
- **Weight decay:** $\alpha \approx 1.325 \times 10^{-5}$.
 - **Learning rate:** $\eta \approx 8.053 \times 10^{-4}$.
 - **Best validation Macro-F1: 0.9106.**

The parameter-importance analysis produced by Optuna identified the *learning rate* and *first-layer width* as the most influential hyperparameters, while the weight-decay strength and depth contributed comparatively less on this dataset. The optimisation-history curve plateaus well before trial 100, providing empirical evidence that the trial budget was sufficient: additional trials would not meaningfully improve the discovered configuration.

Statistical Stability: 5-Seed Verification

To confirm that the optimised configuration is not a product of a fortunate random initialisation, we retrained the best architecture with five different seeds $\{42, 5435, 123, 2026, 23\}$ and recorded all metrics on the validation set. The resulting aggregate is **0.9031 $\pm$ 0.0024** for Macro-F1 and **0.9034 $\pm$ 0.0021** for accuracy, with a mean training time of 16.6 ± 3.1 seconds. The negligible standard deviation ($\sigma < 0.003$) mathematically confirms the stability of the learned configuration across stochastic training.

Diagnostic and Interpretability Suite

To evaluate the behaviour of the trained MLP, the following diagnostic tools were produced:

1. **Optuna Optimization History and Parameter Importance Plots:** to document search convergence and hyperparameter sensitivity.
2. **Confusion Matrix:** a per-class breakdown of predictions versus true labels to localise systematic errors.
3. **Class-wise F1 Bar Chart:** a ranked view of per-class performance, making the upper-body confusion cluster immediately visible.

CHAPTER 4

Experimental Setup

This chapter describes the experimental framework used to ensure fair evaluation and reproducible results.

4.1. Data Split Strategy

The Fashion-MNIST dataset was divided according to a standard protocol to support rigorous model development and unbiased evaluation. In this project, we adopted a three-way split complemented by a repeated hold-out validation framework:

- **Training Set (54,000 samples):** From the original 60,000-sample training partition, 90% of the images were used for model fitting. This subset was also used to compute the parameters required for feature extraction and statistical normalization.
- **Validation Set (6,000 samples) and Repeated Hold-out Strategy:** A 10% hold-out portion of the original training partition was created using a *stratified split* to preserve the class distribution across the ten clothing categories. During hyperparameter optimization with Bayesian search in Optuna, a fixed random seed of 42 was used to provide a consistent validation benchmark. After optimization, all models were further evaluated using a **five-seed repeated hold-out validation** scheme to assess stability and reduce the risk of overfitting to a single split. This procedure involved creating five independent stratified train-validation splits from the original 60,000 samples. The resulting metrics, such as accuracy and macro-F1, were then aggregated and reported as the mean and standard deviation ($\mu \pm \sigma$).
- **Testing Set (10,000 samples):** This set remained completely untouched during feature engineering, model selection, hyperparameter tuning, and stability evaluation. It served as a strict, single-use benchmark for reporting final generalization performance and inference

latency on unseen data.

4.2. Evaluation Protocol and Metrics

All four models—Logistic Regression, MLP, Random Forest, and SVM—were evaluated under a unified protocol to ensure a fair and comparable assessment. Each model was first tuned on the internal validation split, while the held-out test set was reserved for a single final evaluation to prevent information leakage. To assess the reliability of the results, each model also underwent a **five-seed statistical stability check**, in which the best hyperparameter configuration was re-evaluated across five different random seeds and the mean (μ) and standard deviation (σ) of each metric were reported.

The following metrics were computed consistently for all models (where TP , FP , and FN denote True Positives, False Positives, and False Negatives, respectively):

- **Accuracy:** The proportion of correctly classified instances among all $N = 10,000$ test samples.

$$\text{Accuracy} = \frac{\sum_{i=1}^K TP_i}{N} \quad (4.1)$$

where $K = 10$ is the number of classes. Because Fashion-MNIST is a balanced dataset, accuracy serves as the primary performance indicator.

- **Macro-Averaged Precision, Recall, and F1-Score:** These metrics are first calculated per class i and then averaged uniformly across all categories.

- *Precision* measures the exactness of the classifier: $P_i = \frac{TP_i}{TP_i + FP_i}$
- *Recall* measures the completeness of the classifier: $R_i = \frac{TP_i}{TP_i + FN_i}$
- *F1-Score* is the harmonic mean of precision and recall: $F1_i = 2 \cdot \frac{P_i \cdot R_i}{P_i + R_i}$

The final **Macro-F1** is computed as:

$$F1_{macro} = \frac{1}{K} \sum_{i=1}^K F1_i \quad (4.2)$$

Macro-averaging was preferred because it highlights weaknesses in individual classes without the masking effect of weighted averaging.

-
- **Confusion Matrix:** A 10×10 matrix $\mathcal{C}$ where each element $C_{i,j}$ represents the number of observations known to be in group i but predicted to be in group j . This qualitative diagnostic helps identify systematic confusion between visually similar categories.

- **Inference Latency:** The average time t_{inf} required to classify a single image, computed over the test set $\mathcal{D}_{test}$:

$$t_{inf} = \frac{\text{Total Inference Time}}{|\mathcal{D}_{test}|} \quad (4.3)$$

- **Training Time:** The total wall-clock time required to converge on the training set. Together with latency, this captures the computational cost of each approach.

For the stability analysis, each metric M was summarized as $\mu_M \pm \sigma_M$ across the five seeds:

$$\mu_M = \frac{1}{5} \sum_{s=1}^5 M_s, \quad \sigma_M = \sqrt{\frac{1}{5} \sum_{s=1}^5 (M_s - \mu_M)^2} \quad (4.4)$$

where a low σ indicates that the model’s performance is robust to variations in data splitting and initialization.

4.3. Hyperparameter Selection Method

All four models—Logistic Regression, MLP, Random Forest, and SVM—use a unified and automated hyperparameter optimization strategy based on **Bayesian optimization** with the **Optuna** framework.

- **Methodology:** Bayesian optimization was performed using Optuna’s **Tree-structured Parzen Estimator (TPE) sampler**. Unlike standard Bayesian optimization that models the objective function $P(y|x)$ using Gaussian Processes, TPE models the likelihood $P(x|y)$ by maintaining two density functions:

$$p(x|y) = \begin{cases} \ell(x) & \text{if } y < y^* \\ g(x) & \text{if } y \geq y^* \end{cases} \quad (4.5)$$

where y^* is a quantile of the observed scores. By maximizing the Expected Improvement (EI), which is proportional to the ratio $\ell(x)/g(x)$, the sampler identifies regions of the

hyperparameter space most likely to yield "good" results ($\ell(x)$) while avoiding "bad" regions ($g(x)$). This approach is theoretically more efficient than Grid Search or Random Search as it transitions from exploration to exploitation as the number of trials increases. To manage computational resources, a **MedianPruner** was integrated. Grounded in the principle of early stopping, it terminates trials whose intermediate results (validation accuracy after n epochs) fall below the median of previous trials at the same step, allowing the budget to be redirected toward more promising configurations.

- **Optimization Goal:** All studies were configured to **maximize the macro-averaged F1-score** ($F1_{macro}$) on a held-out validation set. Theoretically, the choice of the harmonic mean in the F1-score:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4.6)$$

provides a more conservative estimate of performance compared to the arithmetic mean, as it is heavily penalized by low values in either precision or recall. In the context of Fashion-MNIST, while the classes are balanced, certain categories (e.g., *Shirt* vs. *T-shirt/top*) exhibit high semantic similarity. Optimizing for the macro-average ensures that the hyperparameter search does not "sacrifice" the performance of these difficult classes to gain marginal improvements in easier classes, thereby enforcing a model that generalizes uniformly across the entire label space.

- **Search Space:** A continuous or discrete search space was defined for the key hyperparameters of each model:
 - **Logistic Regression:** Regularization strength $C \in [10^{-2}, 10^2]$ (log-uniform), with 15 trials.
 - **MLP:** Learning rate $\eta \in [10^{-4}, 10^{-2}]$, weight decay $\lambda \in [10^{-5}, 10^{-1}]$, number of hidden layers $\in \{1, 2, 3\}$, and units per layer $\in [32, 512]$; 100 trials were conducted with early stopping via **MedianPruner**.
 - **Random Forest:** Number of estimators $\in [800, 1200]$, tree depth $\in [30, 60]$, minimum samples per split $\in [2, 8]$, and feature fraction $\in [0.15, 0.40]$; up to 50 trials were performed.
 - **SVM:** Regularization parameter $C \in [10^{-1}, 10^2]$ (log-uniform) and kernel coefficient $\gamma \in \{\text{scale}, \text{auto}\}$; 20 trials were performed with **MedianPruner**.

4.4. Technical Environment

4.4.1. Software and Libraries

All four models were executed on **Kaggle Notebooks** using Python 3.12. The shared software stack is as follows:

- **Language:** Python 3.12
- **Core ML:** `scikit-learn` (model utilities, preprocessing, metrics), `cuML` (GPU-accelerated Logistic Regression, Random Forest, and SVM via RAPIDS)
- **Deep Learning:** `PyTorch` (MLP model definition, training loop, and GPU tensor ops)
- **Hyperparameter Optimization:** `Optuna` (Bayesian search with `TPESampler`)
- **Feature Engineering:** `scikit-image` (HOG), `Kornia` (GPU-accelerated HOG for Random Forest)
- **Data Manipulation:** `numpy`, `pandas`, `cuDF` (GPU DataFrames for Random Forest)
- **Visualization:** `matplotlib`, `seaborn`, `plotly`

4.4.2. Hardware and Runtime Environment

All experiments were conducted on **Kaggle’s free GPU tier**. The hardware configuration, as reported by the runtime environment probes within the notebooks, was consistent across all four pipelines:

- **Processor (CPU):** Intel(R) Xeon(R) CPU @ 2.00GHz.
 - *Configuration:* 2 physical cores with Hyper-Threading (4 vCPUs).
 - *Role:* Orchestrated the execution of GPU kernels, managed the I/O for the Fashion-MNIST dataset, and handled the high-level logic for the Optuna search trials.
- **Accelerator (GPU):** NVIDIA Tesla T4 (16 GB GDDR6 VRAM).
 - *Architecture:* Turing™ (TU104) with 2,560 CUDA cores and 320 Tensor cores.

-
- *Libraries*: Leveraged `cuML` for GPU-accelerated classical models (Logistic Regression, Random Forest, SVM) and `PyTorch CUDA` for the MLP and HOG feature extraction.

- **System Memory**: ≈ 31.2 GB RAM (LPDDR4).

4.4.3. Reproducibility and Random Seed

To guarantee that all results reported in Chapter 5 can be exactly replicated, a **global random seed of 42** was applied uniformly across all four models and all stochastic operations:

- **Seed value**: `RANDOM_SEED = 42` is declared as a top-level constant in every notebook.
- **Coverage**: The seed is passed to all sources of randomness, including:
 - Stratified data splitting (`train_test_split(..., random_state=42)`)
 - Optuna study initialization (`TPESampler(seed=42)`)
 - Random Forest tree construction (`random_state=42`)
 - MLP weight initialization and data loading (`torch.manual_seed(42)`, `torch.cuda.manual_seed_all`)
 - NumPy global state (`np.random.seed(42)`)
- **Stability verification**: Beyond the fixed seed, all four models additionally perform a **5-seed repeated evaluation** (seeds $\in \{42, 5435, 123, 2026, 23\}$) to report performance as mean $\pm$ standard deviation, confirming that results are stable and not an artifact of a single lucky initialization. (geÓÁron2017hands-on)

CHAPTER 5

Analysis Results

This chapter provides a comprehensive evaluation of the four classification models. The discussion begins with a high-level quantitative comparison, followed by a qualitative assessment of model behavior and a final justification for the most appropriate model for the Fashion-MNIST classification task.

5.1. Qualitative & Quantitative Analysis

5.1.1. Support Vector Maching (SVM)

The final evaluation on the 10,000-sample test set provides a clear numerical breakdown of both the model’s predictive power and its computational efficiency.

Classification Performance

The optimized SVM achieved an Accuracy of 0.9000 and a Macro-F1 score of 0.8999 on unseen data. More importantly, the near-perfect alignment between Macro-Precision (0.8999) and Macro-Recall (0.9000) indicates that the model is highly stable across all 10 categories. It does not inflate its accuracy by over-predicting majority classes, nor does it ignore harder-to-classify items. Hitting this 90% threshold using classical machine learning on the complex Fashion-MNIST dataset validates the effectiveness of the PCA and RBF kernel combination.

Computational Efficiency

Thanks to GPU acceleration, the model processed and trained on the entire 60,000-sample dataset in just 4.1778 seconds. Looking at deployment viability, the inference speed is exceptionally fast, recording an average latency of only 0.0697 ms per image. This sub-millisecond prediction capability confirms that the pipeline is highly suitable for real-time classification scenarios.

Optimization History

This scatter plot tracks the objective value (Macro-F1) across all 20 trials. The curve shows a rapid performance jump in the earliest trials before quickly stabilizing and converging near the peak score. This pattern proves that the defined search space was appropriate and that the Bayesian sampler effectively located the optimal region without wasting computational resources on dead ends.

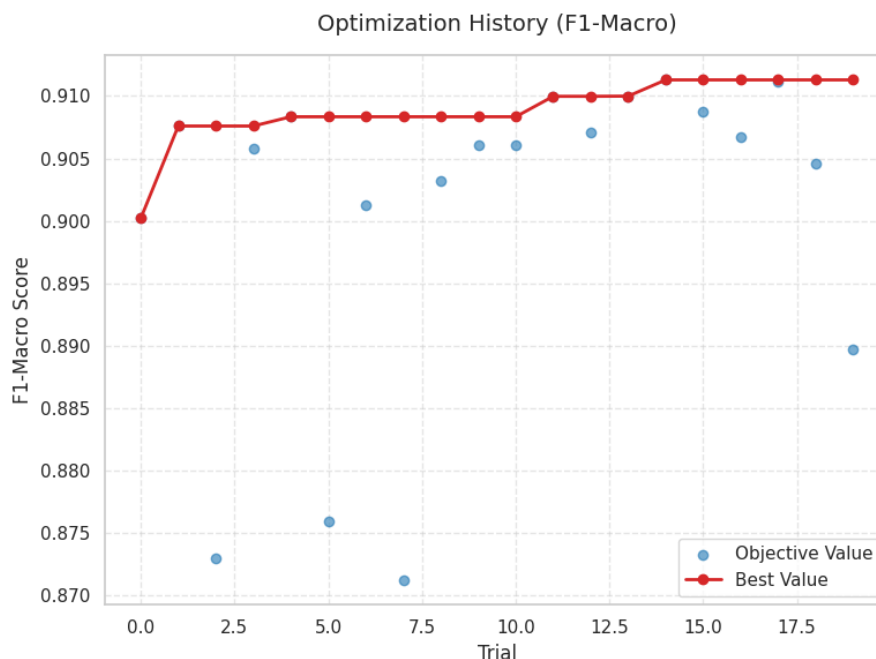


Figure 5.1: Optimization History

Hyperparameter Importance

The fANOVA bar chart isolates the exact contribution of each parameter. It clearly illustrates that the regularization parameter (C) was the absolute driving force behind the performance gains. Meanwhile, the kernel coefficient (γ) had a negligible impact on the final outcome. This insight mathematically justifies the final configuration and shows exactly which variable dictates the SVM's behavior.

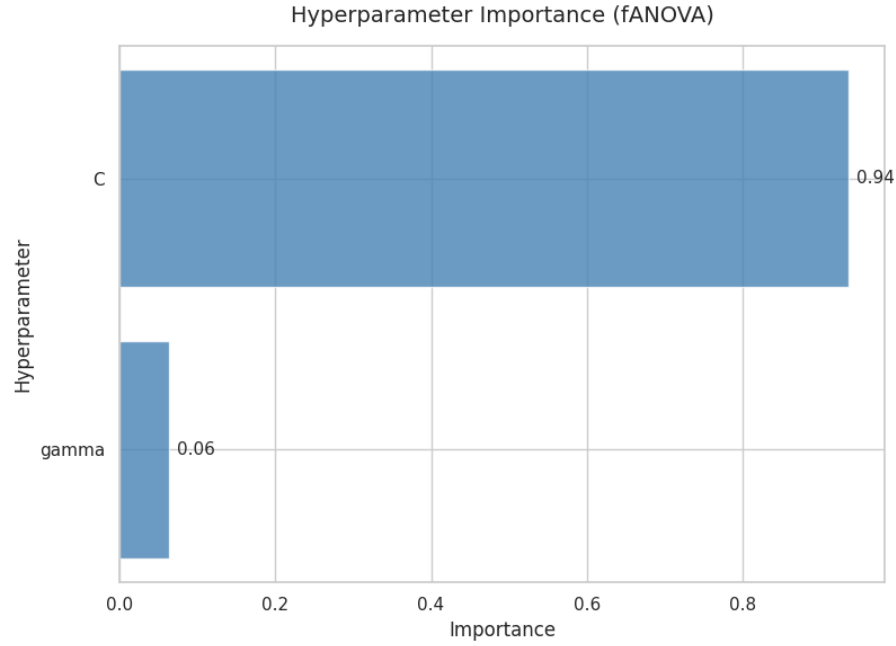


Figure 5.2: Hyperparameter Importance

Statistical Stability (5-Seed Repeated Hold-out)

To verify the model’s robustness against favorable data splits, the optimized pipeline was evaluated across five random seeds. As detailed in the table below, the results demonstrate exceptional stability: the Macro-F1 score averaged 0.9040 with a minimal standard deviation of ± 0.0050 , while training times remained perfectly consistent at 3.87 seconds ($\pm 0.017s$). This extremely tight variance mathematically guarantees that the SVM successfully learned the underlying structural patterns of the dataset rather than exploiting a specific train-validation split. Ultimately, it confirms that the hyperparameters selected by Optuna generalize well, making the 90% accuracy a highly reliable and reproducible baseline.

Run (Seed)	Accuracy	Precision	Recall	Macro-F1	Train Time (s)
Run 1 (42)	0.9115	0.9112	0.9115	0.9113	3.89
Run 2 (5435)	0.9062	0.9055	0.9062	0.9056	3.88
Run 3 (123)	0.9052	0.9054	0.9052	0.9051	3.84
Run 4 (2026)	0.9025	0.9020	0.9025	0.9022	3.87
Run 5 (23)	0.8960	0.8958	0.8960	0.8958	3.88
Aggregate ($\mu \pm \sigma$)	0.9043 $\pm$ 0.005	0.9040 $\pm$ 0.005	0.9043 $\pm$ 0.005	0.9040 $\pm$ 0.005	3.8691 $\pm$ 0.017

Table 5.1: 5-Seed Repeated Hold-out evaluation at optimal $C \approx 17.14$.

While the global metrics indicate a highly accurate model, an aggregate score of 90% often masks underlying class imbalances. To truly understand the SVM’s behavior, its decision boundaries, and its blind spots, a deep dive into the visual diagnostic tools is necessary.

Semantic Ambiguity

The Confusion Matrix perfectly maps out exactly where the model is bleeding accuracy. The errors are not random noise; they are highly localized and semantically logical. For example, actual "Shirts" are consistently misclassified as "T-shirt/top" (100 times), "Pullover" (70 times), and "Coat" (57 times). Similarly, "Coats" and "Pullovers" are repeatedly confused with each other (78 mutual misclassifications). In a 28x28 grayscale environment, the pixel differences between a long-sleeve shirt, a light sweater, and a thin jacket are incredibly subtle. The SVM is capturing the overall shape (long sleeves, torso coverage) but failing to isolate fine-grained details like collars, buttons, or zipper lines.

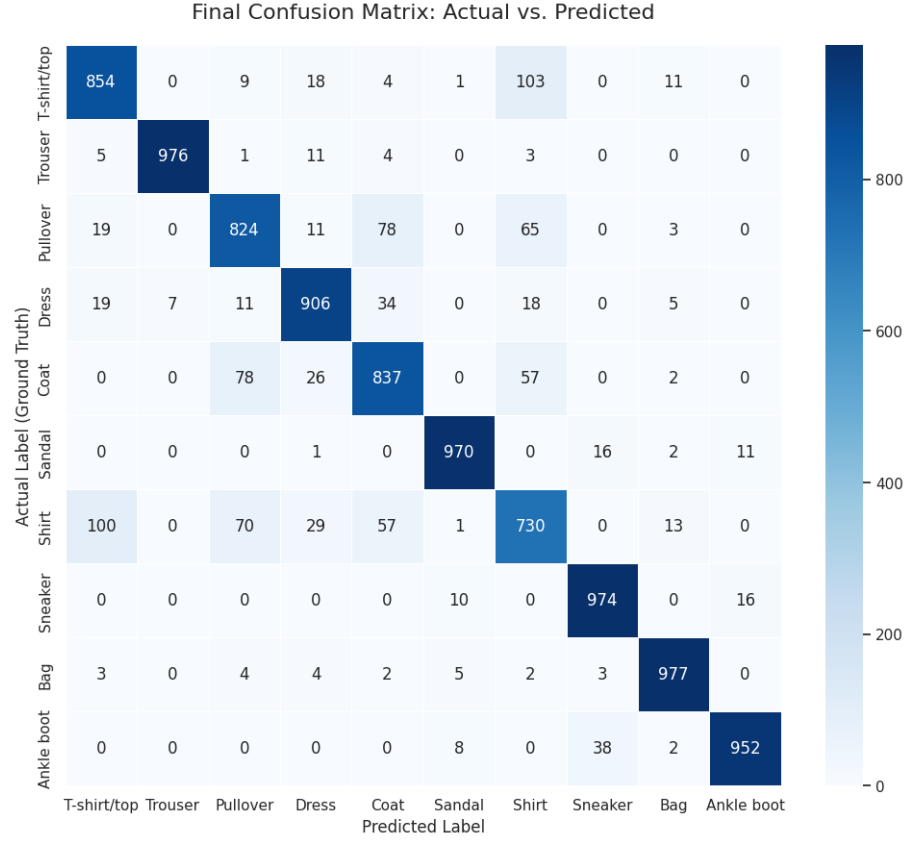


Figure 5.3: Confusion Matrix

The Complexity Divide

The F1-Score Heatmap shows that the model is excellent at identifying items with clear, distinct shapes. It gets near-perfect scores for categories like "Trouser" (0.984), "Sandal" (0.972), and "Bag" (0.970). On the other hand, it has a much harder time with upper-body clothing. The "Shirt" category is the weakest point of the model, scoring only 0.738, followed by "Pullover" (0.825) and "Coat" (0.830).

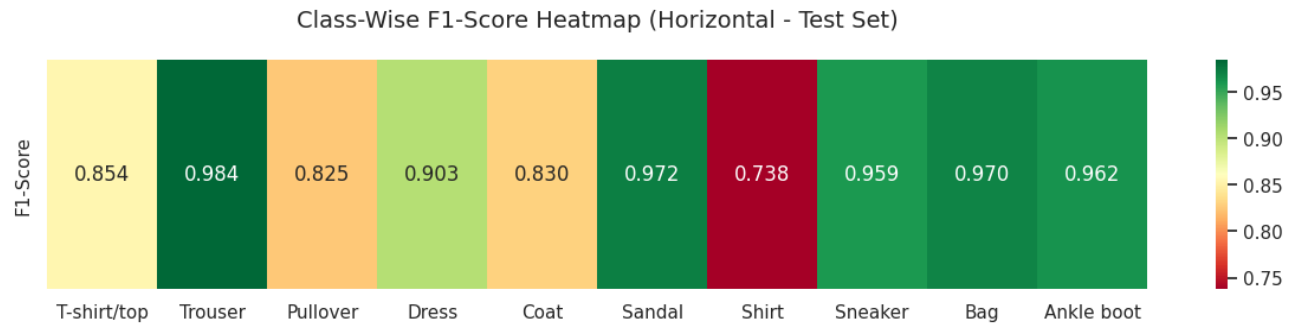


Figure 5.4: F1-Score Heatmap

Visualizing the Errors

Looking at the actual misclassified images makes it clear why these mix-ups happen—even a human might struggle with some of these edge cases.

- A short-sleeved "Shirt" gets labeled as a "T-shirt/top".
- A thick, textured "Coat" is predicted to be a "Pullover".
- Even in the usually accurate shoe categories, a weirdly angled "Sneaker" or "Ankle boot" might get flagged as a "Sandal".

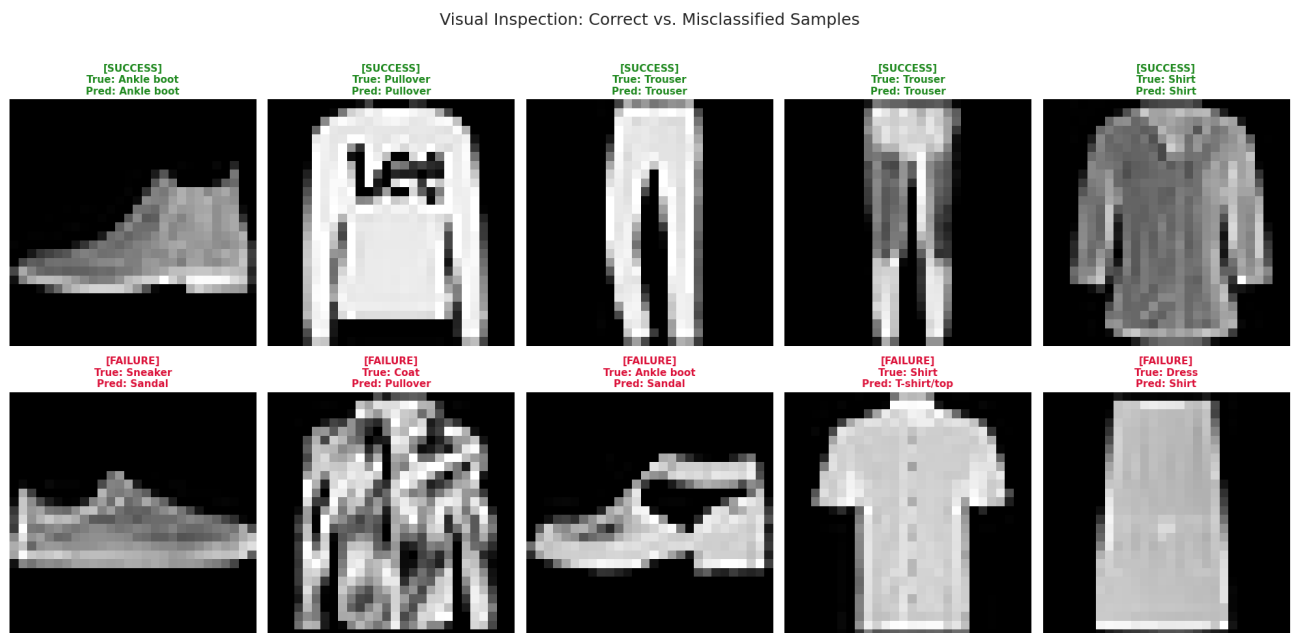


Figure 5.5: Error Grids: Success vs Failure

Conclusion

Overall, this visual check confirms that the SVM is working properly. It has learned the basic shapes of the clothing items, and its mistakes are mostly limited to tricky edge cases where the low image resolution simply doesn't provide enough detail to tell them apart.

5.1.2. Logistic Regression

Feature Transformation Output (HOG)

The first phase of the pipeline successfully mapped the raw 784-dimensional spatial pixels into 324 structural gradient features. **Figure 5.6** visually demonstrates this transformation, confirming that the HOG algorithm effectively discarded raw illumination variations in favor of highlighting critical object contours and vertical/horizontal edge orientations.

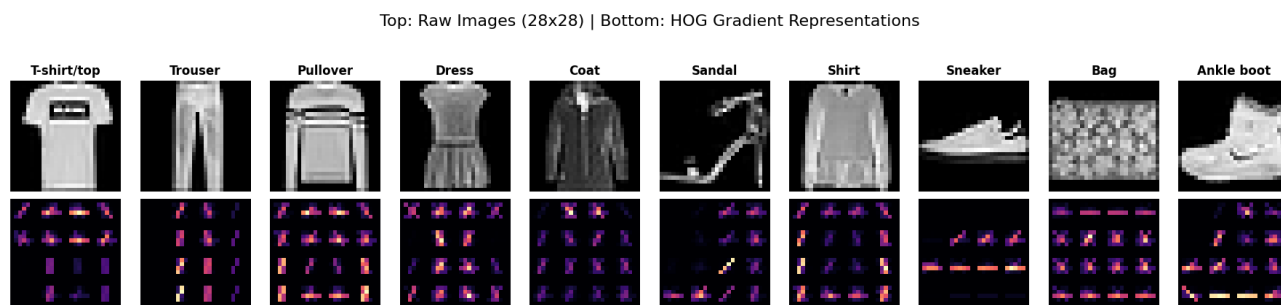


Figure 5.6: Visual output of the HOG transformation: Raw spatial pixels (left) vs. Gradient representations (right).

Hyperparameter Optimization Results (Optuna)

The Bayesian Optimization process rigorously explored the regularization search space. As shown in **Figure 5.7**, the Tree-structured Parzen Estimator (TPE) successfully minimized the loss landscape, converging on an optimal inverse regularization strength of $C \approx 0.31471$. This configuration yielded a peak Macro-F1 score of **0.8609** on the isolated validation set.

Optuna: Optimization History (Macro-F1 Progress)

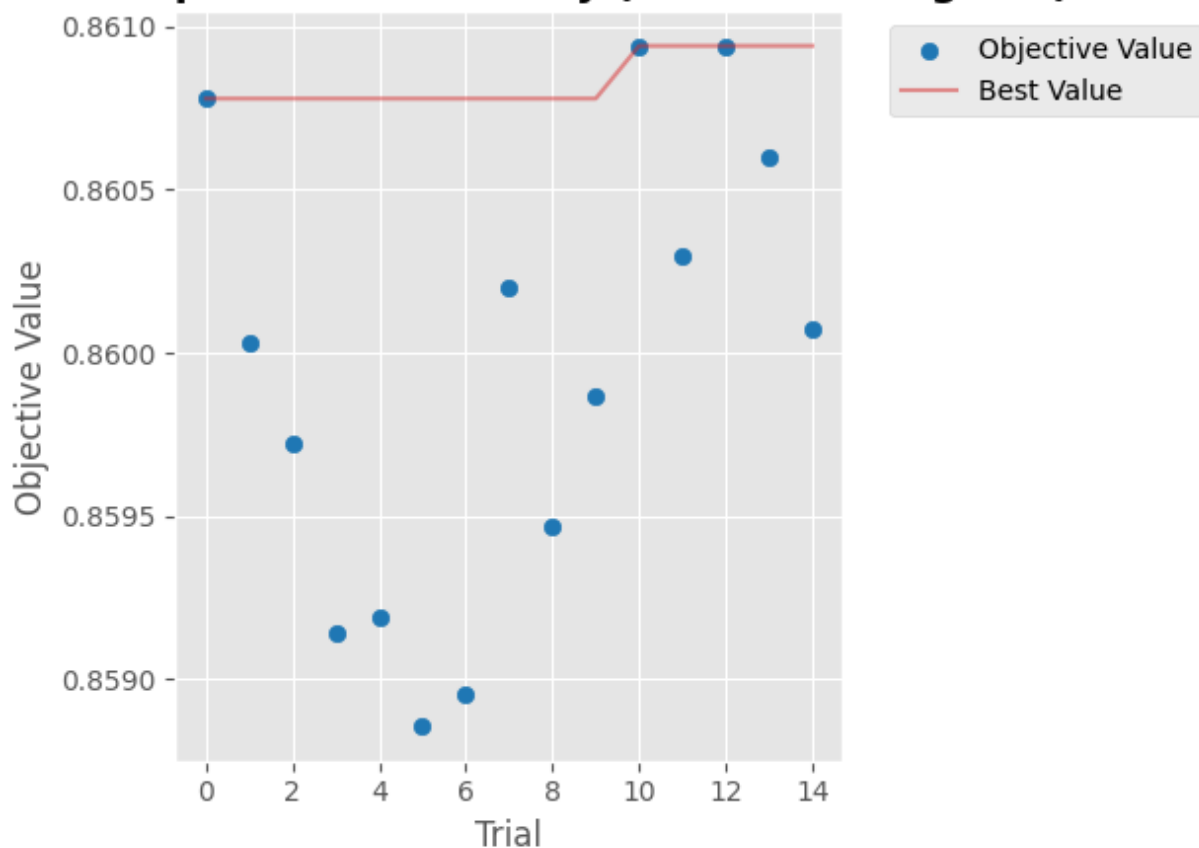


Figure 5.7: Bayesian Optimization history tracking the Macro-F1 score against the regularization parameter (C).

Statistical Stability (5-Seed Repeated Hold-out)

Before finalizing the model, the optimal hyperparameter ($C \approx 0.31471$) was subjected to a 5-seed repeated hold-out test to mathematically rule out data split bias.

Run (Seed)	Accuracy	Precision	Recall	Macro-F1	Train Time (s)
Run 1 (42)	0.8612	0.8605	0.8612	0.8608	1.14
Run 2 (5435)	0.8533	0.8520	0.8533	0.8524	1.07
Run 3 (123)	0.8520	0.8515	0.8520	0.8515	0.86
Run 4 (2026)	0.8562	0.8554	0.8562	0.8556	0.92
Run 5 (23)	0.8548	0.8541	0.8548	0.8543	0.90
Aggregate ($\mu \pm \sigma$)	0.8555 $\pm$ 0.003	0.8547 $\pm$ 0.003	0.8555 $\pm$ 0.003	0.8549 $\pm$ 0.003	0.98 $\pm$ 0.10

Table 5.2: 5-Seed Repeated Hold-out evaluation at optimal $C \approx 0.31471$.

Table 5.2 confirms that the model exhibits exceptional stability. The remarkably low standard deviation ($\sigma \approx 0.003$) across all primary metrics proves that the model’s performance is strictly driven by the learned features rather than randomized dataset splits.

Final Quantitative Evaluation (Test Set)

With stability verified, the model was retrained on the entire 60,000-sample training set. **Table 5.3** presents the absolute performance of the Logistic Regression model on the 10,000 completely unseen test samples.

Evaluation Metric	Performance
Macro-F1 Score	84.54%
Accuracy	84.60%
Macro-Precision	84.53%
Macro-Recall	84.60%
Training Time (60,000 samples)	1.20 seconds
Inference Latency	0.0015 ms/image

Table 5.3: Final quantitative performance of the optimized Logistic Regression on the Test Set.

Qualitative Error Analysis (Confusion Matrix)

To extract algorithmic insights beyond raw numbers, we performed a structural analysis of the final confusion matrix (**Figure 5.8**).

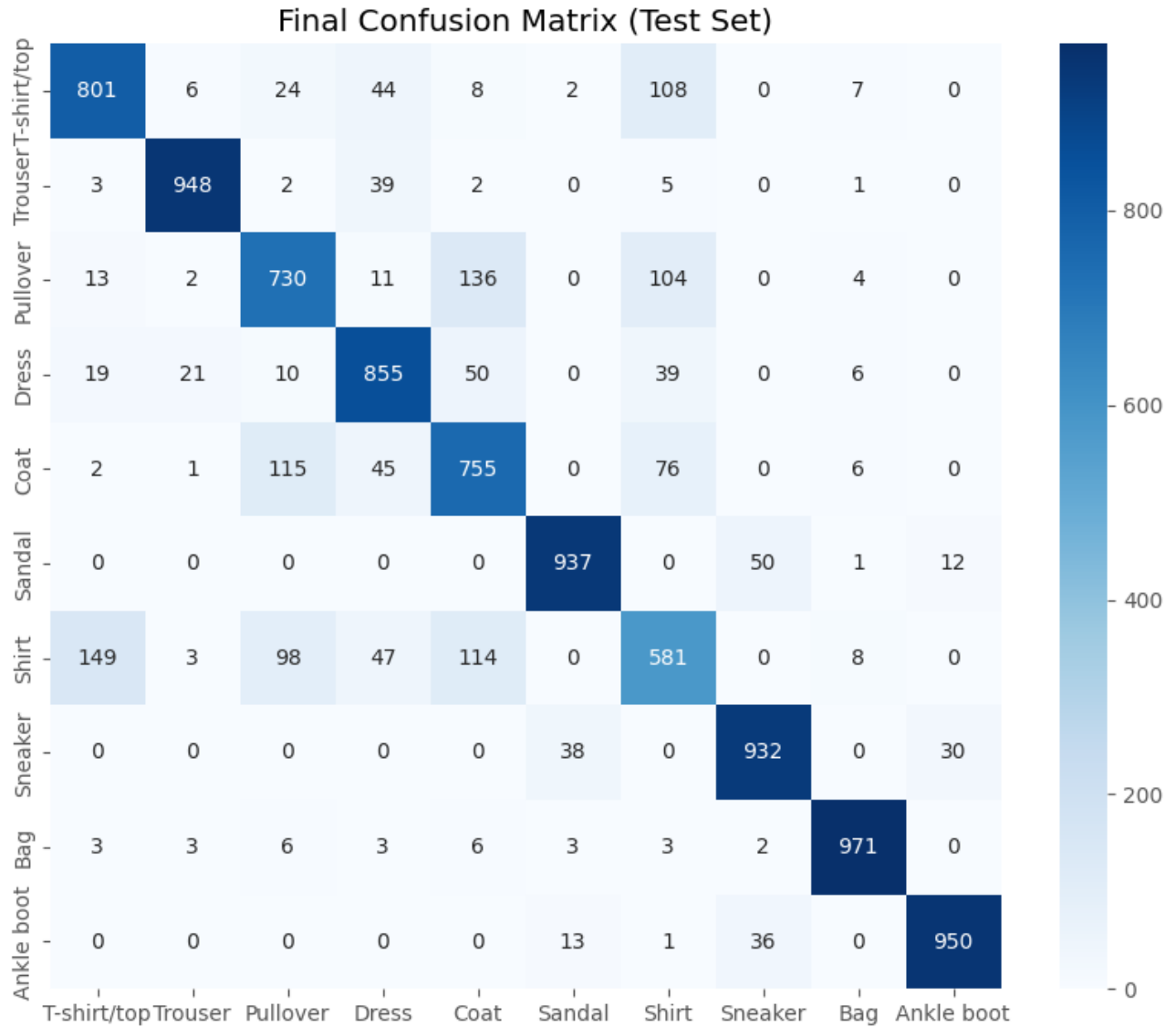


Figure 5.8: Final Confusion Matrix for the Logistic Regression Model on the Test Set.

5.1.3. Random Forest

Global and Class-Wise Performance Metrics

To evaluate the model’s objective performance, a class-wise F1-score analysis was conducted on the test set. The distribution of these metrics (Fig. 5.9) provides a numerical baseline for the model’s discriminative power.

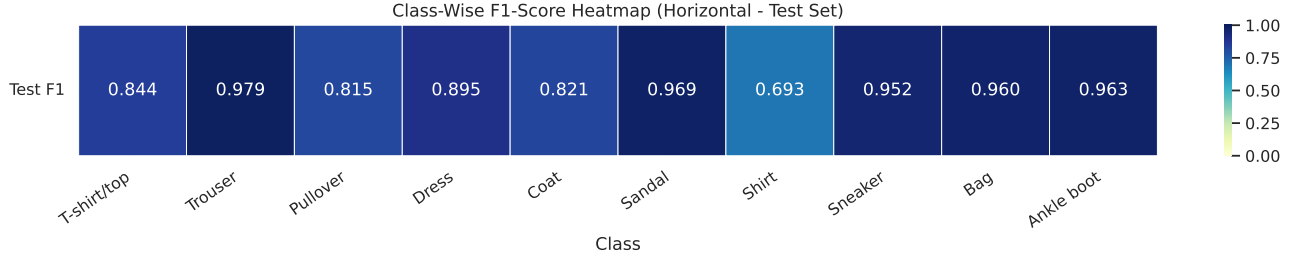


Figure 5.9: Class-wise F1-score heatmap illustrating objective performance across the ten categories.

The scores partition the dataset into three distinct numerical tiers:

- **Tier 1 ($F1 > 0.95$):** *Trousers* (0.979), *Sandal* (0.969), *Ankle boot* (0.963), *Bag* (0.960), and *Sneaker* (0.952).
- **Tier 2 ($0.80 < F1 < 0.90$):** *Dress* (0.895), *T-shirt/top* (0.844), *Coat* (0.821), and *Pullover* (0.815).
- **Tier 3 (Critical Bottleneck):** *Shirt* (0.693).

Quantitative Interpretation: The high precision in Tier 1 suggests that the feature extractor produces highly separable clusters for non-upper-body items. However, the **30% performance gap** between *Trousers* and *Shirt* indicates a significant loss of discriminative information within the upper-body manifold, where HOG descriptors likely fail to capture fine-grained variance.

Error Analysis via Confusion Matrix

To visualize the interaction between classes, a confusion matrix was generated (Fig. 5.10). The diagonal elements represent the recall for each class, while off-diagonal values pinpoint specific inter-class "leakage."

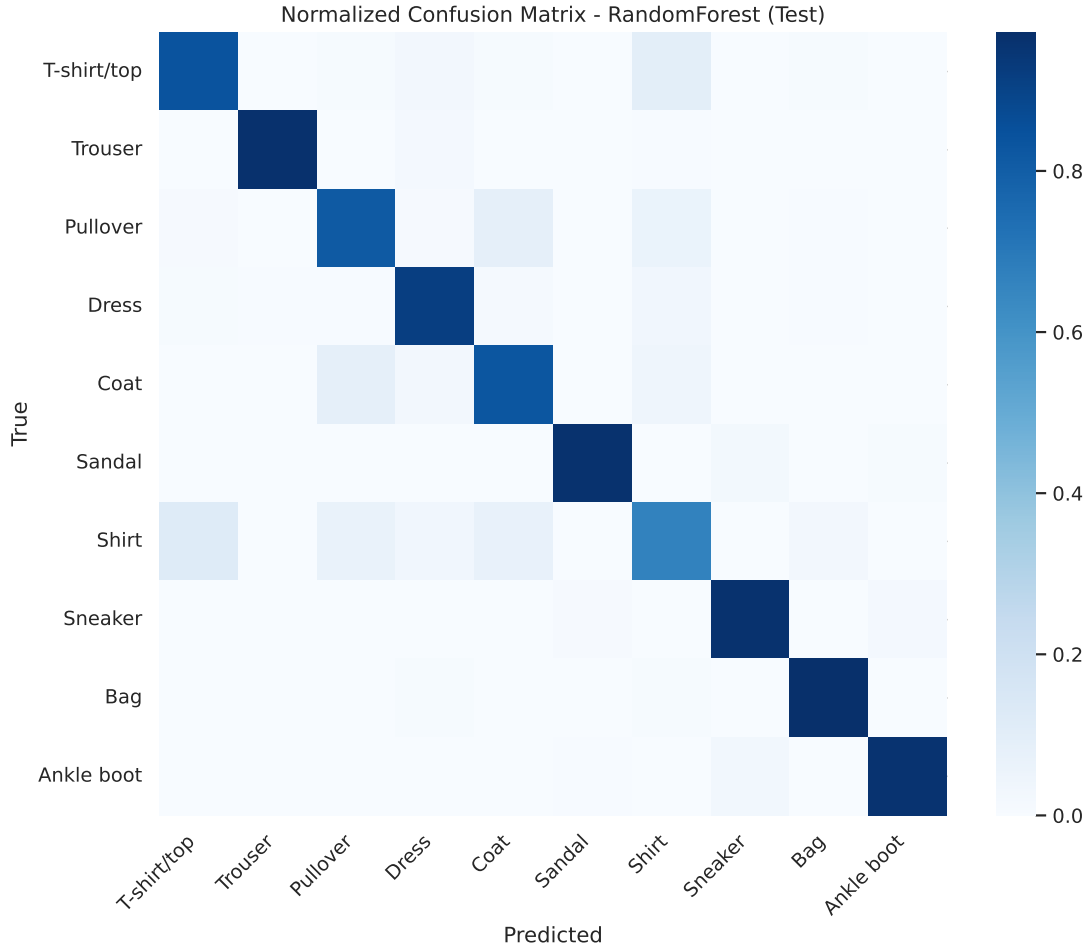


Figure 5.10: Confusion matrix of the Random Forest model on the test set. Rows represent true labels; columns represent predictions.

Quantitative Interpretation: The matrix reveals a highly localized **"Garment Confusion Block"** among indices 0, 2, 4, and 6.

- **The Shirt/Coat/Pullover Cluster:** The *Shirt* class is the most volatile, leaking heavily into *Coat*, *Pullover*, and *T-shirt/top*. This indicates that the PCA-reduced HOG features effectively capture the "torso-plus-arms" geometry but lose the high-frequency edge data (collars, buttons) needed to differentiate these garments.
- **Structural Distinctness:** *Trouser* and *Bag* achieve near-perfect scores, confirming that their aspect ratios and closed-loop silhouettes are mathematically unique within the learned feature space.

- **Footwear Cross-Talk:** Minor confusion exists between *Ankle boot* and *Sneaker*, suggesting that at 28×28 resolution, the "height" of the ankle wrap is a subtle discriminative bin that is easily obscured by variance in shoe style.

Dimensionality and Optimization Data

Dimensionality reduction was quantitatively guided by the cumulative explained-variance curve (Fig. ??). We selected $k = 110$ components, which retains exactly **29% of the total variance**.

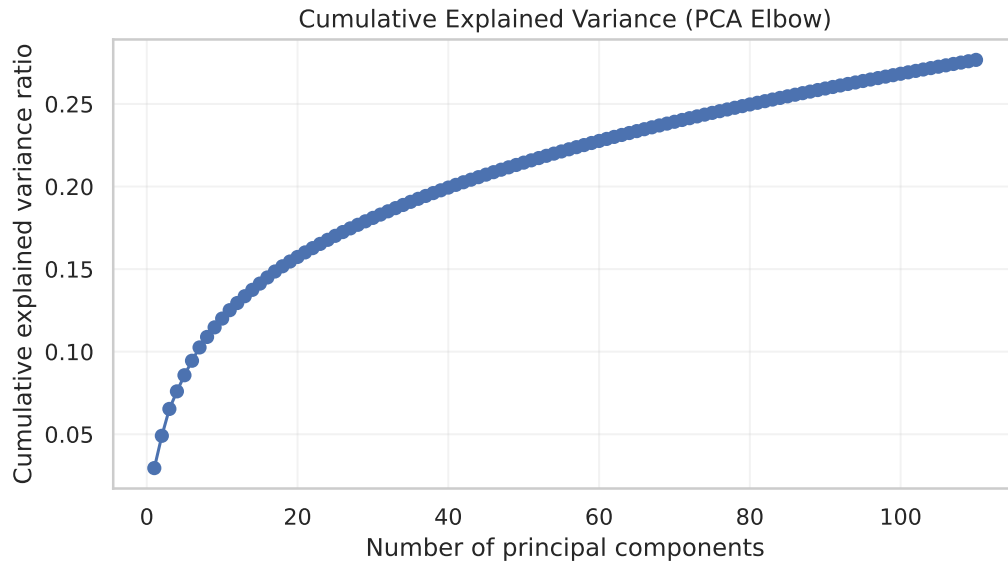


Figure 5.11: PCA cumulative explained-variance curve showing the elbow at $k = 110$.

This threshold was selected by identifying the "elbow" point in the variance curve (Fig. 5.11), a heuristic used to balance computational efficiency with descriptive power. This methodological choice aligns with a practical approach to computer vision that prioritizes basic techniques capable of functioning under real-world conditions rather than pursuing exhaustive mathematical reconstruction. By focusing on this specific variance window, the model isolates primary geometric features—specifically edges and structural lines—and subjects them to a rigorous validation strategy to ensure the resulting decision boundaries are robust and not artifacts of high-frequency noise.

Quantitative Interpretation: While 29% variance appears low for traditional image reconstruction, in the context of this Random Forest pipeline, this acts as an **intentional information**

bottleneck. By discarding 71% of the variance—which likely consists of high-frequency pixel noise and fabric-level irregularities—the model is forced to learn decision boundaries based only on the most stable, global geometric structures.

The hyperparameter space was explored using Bayesian optimization via the Optuna framework. The search converged at **Trial 13** with a **best Macro-F1 score of 0.898085**. Table 5.4 details the final optimal configuration.

Table 5.4: Best hyperparameters obtained at Trial 13.

Hyperparameter	Optimal Value
n_estimators	1,068
max_depth	48
min_samples_split	5
max_features	0.2572

Quantitative Interpretation: The requirement for a high `max_depth` (48) indicates that the latent space formed by the PCA components is highly non-linear, necessitating deep recursive partitioning to isolate distinct silhouette clusters. The high number of estimators (1,068) is statistically necessary to stabilize the high variance inherent in such deep trees, ensuring a robust ensemble vote.

Statistical Stability and Robustness Check

To ensure the results were not artifacts of a specific initialization, the model underwent a five-seed re-evaluation across a range of random states.

Table 5.5: Robustness metrics across five independent seeds (mean $\pm$ std).

Metric	Value (Valid)
Accuracy	0.8946 ± 0.0040
Precision	0.8939 ± 0.0039
Recall	0.8946 ± 0.0040
Macro-F1	0.8940 ± 0.0040
Training Time (sec)	22.0018 ± 0.0717

Quantitative Interpretation: The standard deviation of **0.0040** represents a relative variation of only **0.45%** from the mean. This confirms that the performance is driven by the learned structural patterns of the Fashion-MNIST dataset rather than random seed artifacts. Furthermore, the low variance in training time underscores the deterministic efficiency of the GPU-native environment used for this study.

Data Scalability and Learning Dynamics

To assess how the Random Forest model scales with increasing data volume, a learning curve was generated, plotting validation accuracy against the number of training samples (Fig. 5.12). This analysis determines whether the model has reached its asymptotic performance limit or if it remains "data-hungry."

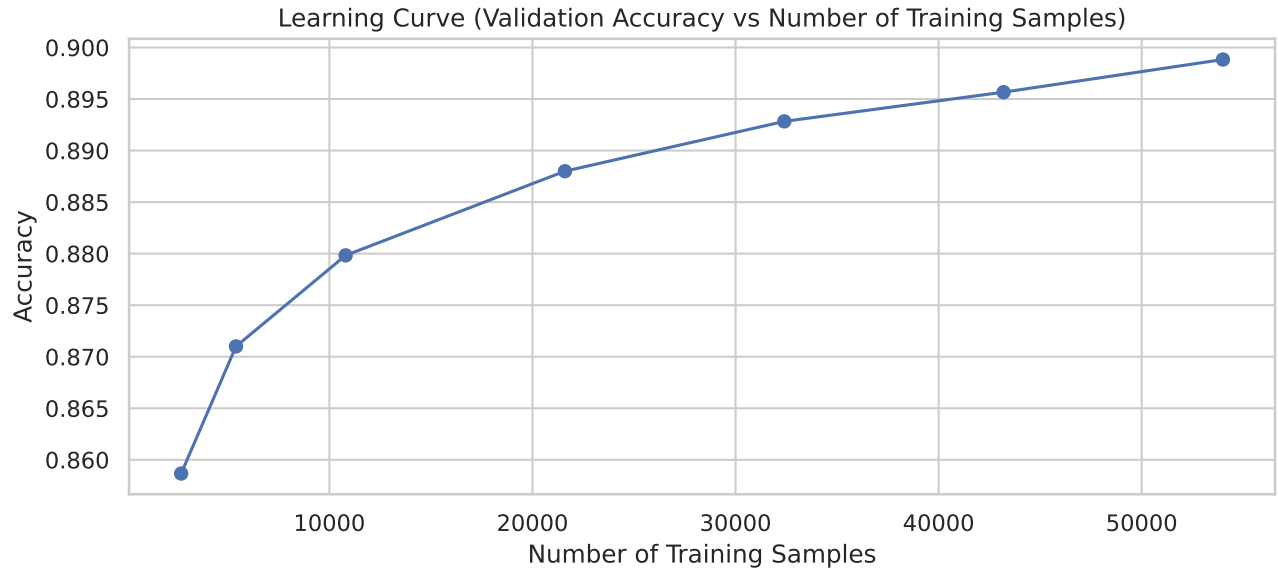


Figure 5.12: Learning curve illustrating the relationship between training set size and validation accuracy.

Quantitative Interpretation: The curve exhibits a classic logarithmic trajectory, revealing three distinct phases of learning:

- **Rapid Acquisition Phase (< 15,000 samples):** The model shows a sharp gain in accuracy, climbing from **0.858** to **approximately 0.885**. In this regime, the Random Forest is rapidly identifying the primary geometric "anchors" (e.g., distinguishing a long *Trouser* silhouette from a compact *Bag*).
- **Refinement Phase (15,000 – 35,000 samples):** The slope begins to moderate. Here, the additional data helps the ensemble stabilize decision boundaries for the "Garment Confusion Block"
- **Asymptotic Convergence (> 40,000 samples):** As the training set approaches 55,000 samples, the accuracy converges toward **0.898**. The diminishing marginal returns suggest that the current feature space (PCA-reduced HOG) is reaching its maximum expressive capacity.

The lack of a total "plateau" at the final data points—indicated by the slight upward tilt between 43,000 and 54,000 samples—suggests that while the model is well-fit, it could potentially

extract marginal gains from an even larger dataset. However, the **4% total accuracy spread** between the smallest and largest training subsets confirms that the model is highly sample-efficient, achieving over 95% of its peak performance with only 20% of the available training data.

Instance-Level Case Studies

To provide a granular understanding of the model’s decision-making failure points, a set of 12 representative misclassifications was analyzed. These errors are categorized into two regimes based on the confidence margin between the top predictions: **Moderate Uncertainty** and **High Ambiguity** (Table 5.6).

Table 5.6: Distribution of failure modes across analyzed samples.

Error Type	Count
Moderate Uncertainty	8
High Ambiguity	4
Total Analyzed	12

The specific instance-level data, including Top-3 class probabilities, is detailed in Table 5.7.

Table 5.7: Detailed instance-level misclassification samples with Top-3 probabilities.

Index	True	Pred	Top-1 (Prob)	Top-2 (Prob)	Top-3 (Prob)	Error Type
6	Coat	Dress	Dress (0.219)	Coat (0.193)	T-shirt/top (0.192)	High Ambiguity
12	Sneaker	Sandal	Sandal (0.613)	Sneaker (0.269)	Bag (0.069)	Moderate Uncertainty
17	Coat	Shirt	Shirt (0.386)	Coat (0.376)	Pullover (0.131)	High Ambiguity
23	Ankle boot	Sandal	Sandal (0.446)	Ankle boot (0.273)	Sneaker (0.264)	Moderate Uncertainty
25	Coat	Pullover	Pullover (0.427)	Shirt (0.339)	Coat (0.151)	Moderate Uncertainty
26	Shirt	Coat	Coat (0.465)	Shirt (0.435)	Pullover (0.090)	High Ambiguity
40	Shirt	T-shirt/top	T-shirt/top (0.877)	Shirt (0.104)	Pullover (0.008)	Moderate Uncertainty
42	Dress	Shirt	Shirt (0.439)	Dress (0.158)	T-shirt/top (0.152)	Moderate Uncertainty
48	Pullover	Coat	Coat (0.360)	Shirt (0.329)	Pullover (0.166)	High Ambiguity
51	Coat	Pullover	Pullover (0.440)	Shirt (0.279)	Coat (0.215)	Moderate Uncertainty
66	Pullover	Dress	Dress (0.661)	Coat (0.152)	Shirt (0.062)	Moderate Uncertainty
68	Ankle boot	Sneaker	Sneaker (0.813)	Ankle boot (0.094)	Sandal (0.083)	Moderate Uncertainty

Quantitative Interpretation: The failure analysis reveals a stark contrast in the model's "confidence" during misclassification.

- **High Ambiguity (33% of samples):** In cases like **Index 6**, the probability distribution is nearly uniform across three categories (*Dress* 0.219, *Coat* 0.193, *T-shirt* 0.192). This numerical "flatness" confirms that the sample resides at a multi-class intersection of the feature space where HOG descriptors cannot isolate a dominant structural signature.
- **Over-Confident Errors:** In **Index 40**, the model exhibits **Moderate Uncertainty** but predicts the wrong class with high confidence ($P = 0.877$). The massive margin (0.773) between the predicted T-shirt and true Shirt suggests a "feature trap" where a specific geometric artifact (e.g., a missing collar silhouette at 28×28 resolution) strongly mimics the prototypical signature of a different class.
- **The Footwear Confusion:** Footwear errors (**Index 12, 23, 68**) consistently show the *Sneaker* or *Ankle boot* as a strong Top-2 contender, indicating that while the model recognizes the "footwear-like" HOG energy, the low-frequency PCA features struggle to differentiate solid texture from open straps.

Feature Abstraction and HOG Pipeline Analysis

The qualitative success of the model is rooted in the transformation of raw pixels into structural energy maps. As seen in Figure 5.13, the **Sobel Gradient Magnitude** effectively suppresses internal texture and noise to isolate the primary silhouette.

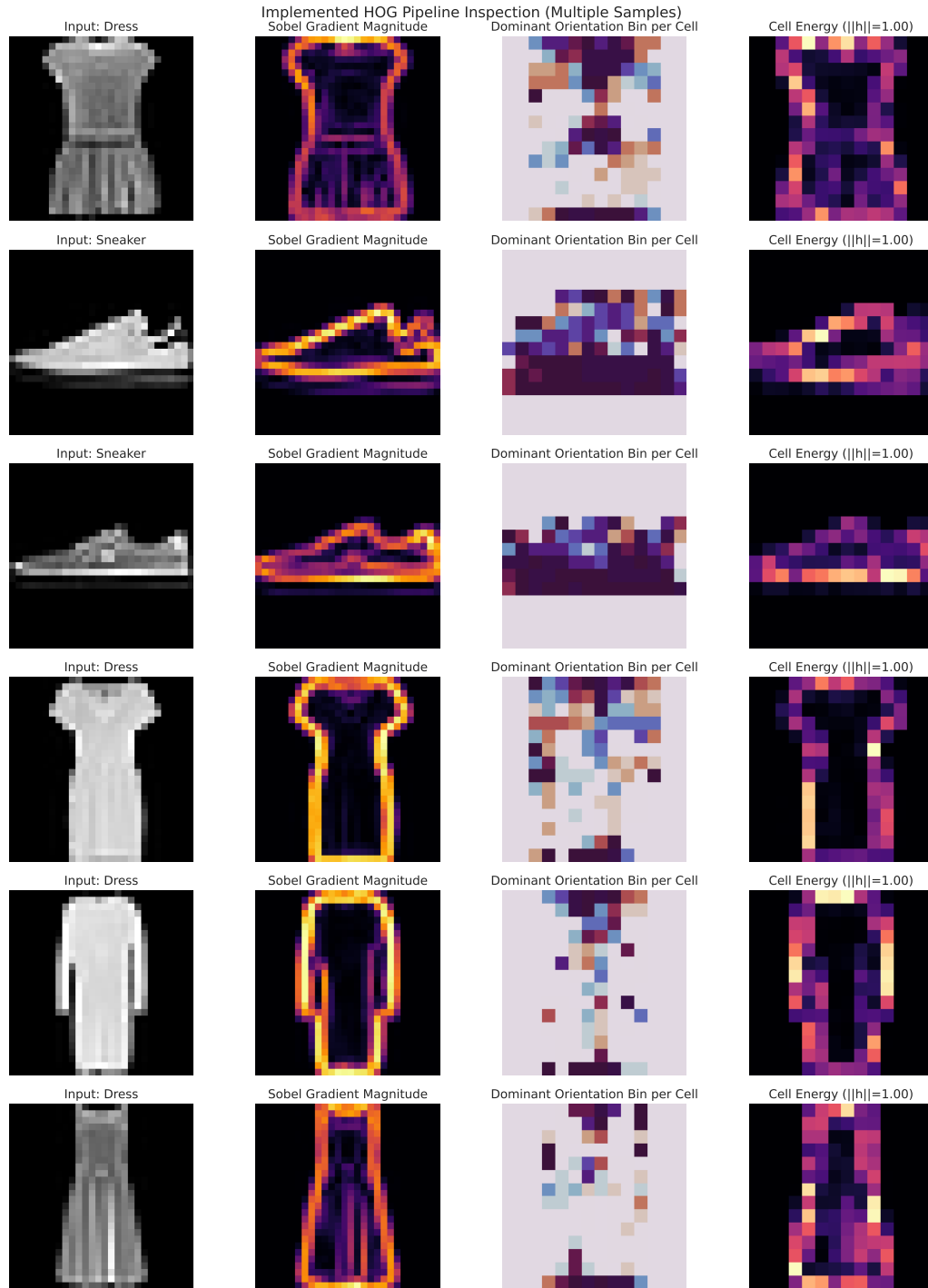


Figure 5.13: HOG pipeline inspection illustrating the abstraction from raw pixels to shape-stable descriptors.

Deep Analysis: The transition to **Dominant Orientation Bins** provides a "shape-stable"

input. Even with minor garment shifts, the localized energy peaks remain consistent, allowing the model to recognize rigid footwear silhouettes more effectively than flexible upper-body garments.

Qualitative Error Analysis and Ambiguity

The confusion matrix reveals a distinct "upper-body confusion block." Qualitatively, this suggests that HOG descriptors effectively capture the general torso silhouette but fail to represent the fine-grained textures—such as buttons, zippers, or knit patterns—required to distinguish a *Shirt* from a *T-shirt* or *Pullover*.

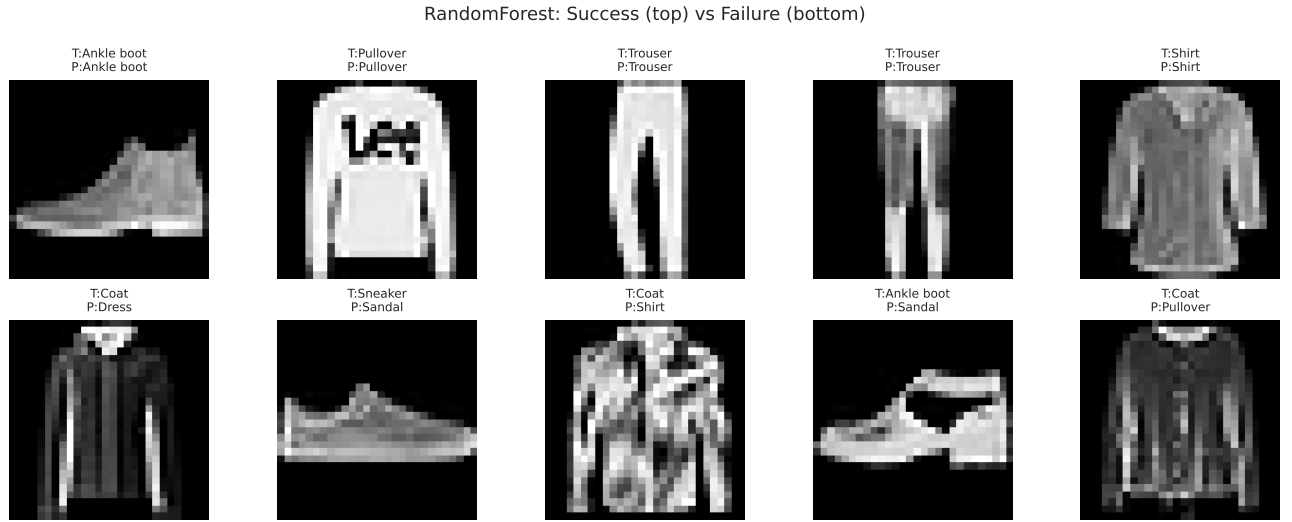


Figure 5.14: Visual inspection: Success cases (top) showing prototypical signatures and failure cases (bottom) showcasing structural ambiguity and feature overlap.

Visual Failure Breakdown Analysis of the failure cases in Figure 5.14 reveals several primary qualitative bottlenecks inherent to the HOG + Random Forest pipeline:

- **Silhouette Overlap and Length Ambiguity:** At a 28×28 resolution, a long open-front *Coat* is often geometrically indistinguishable from a *Dress*. Since HOG features prioritize edge orientations over fabric thickness, the shared flared-bottom silhouette leads to high misclassification rates.
- **Vertical Symmetry and the "Pullover" Sink:** The confusion between *Coats* and *Pullovers* (bottom right) highlights a lack of internal feature hierarchy. The strong vertical gradients of

a coat's opening are often smoothed out during cell normalization, causing the descriptor to mimic the symmetrical signature of a pullover.

- **Aspect Ratio and Neckline Ambiguity:** In *Coat* vs. *Shirt* errors, the "V" shape created by lapels is geometrically similar to a shirt's collar within the top-center spatial bins. Without high-frequency texture data, the model defaults to the class with the most similar orientation energy.
- **Feature Sparsity in Footwear:** In *Ankle boot* vs. *Sandal* errors, side cut-outs or high-contrast reflections create orientation energy peaks that the model interprets as strap-like structures. This suggests a "Heel-to-Strap" mapping error where voids are misinterpreted as structural components.
- **Spatial Quantization Bottleneck:** The confusion between *Sneakers* and *Sandals* suggests that HOG is highly sensitive to object pose. A sneaker at a slight angle produces diagonal edge orientations that mimic the straps of a sandal, revealing a lack of rotational invariance in the feature set.

5.1.4. Multi-layer Perceptron (MLP)

Per-Class Performance

Figure 5.15 shows the confusion matrix of the optimised MLP on the unseen test set, and Figure 5.16 ranks the per-class F1-scores.

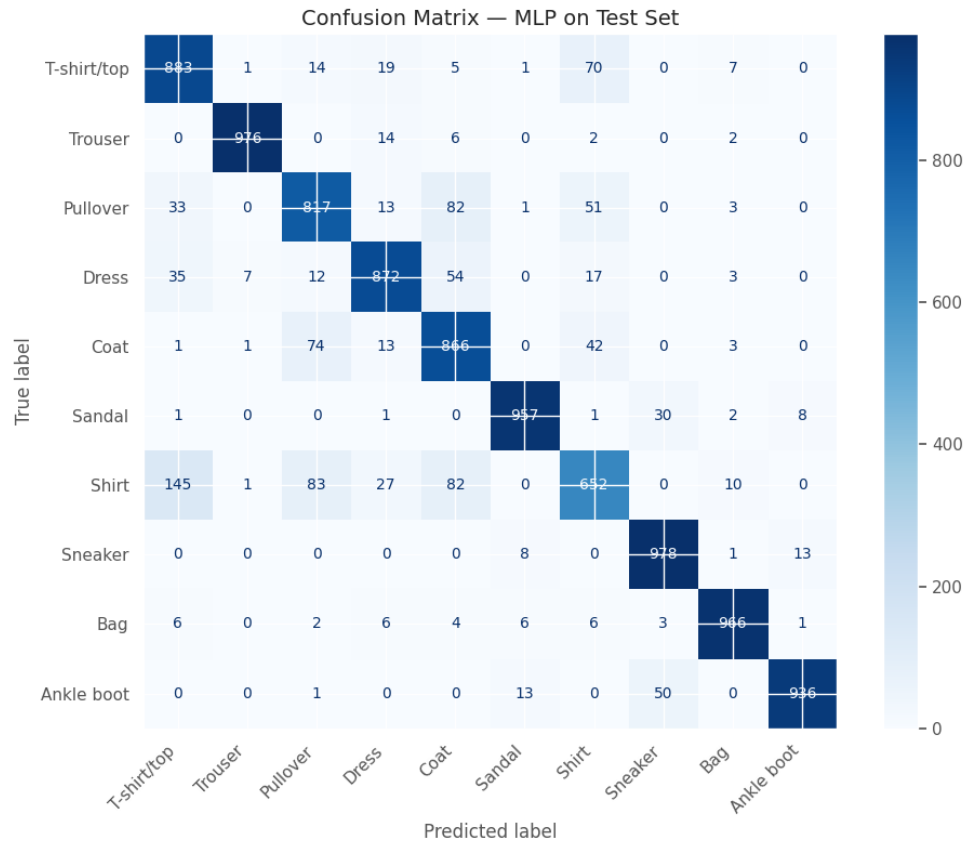


Figure 5.15: Confusion matrix of the optimised MLP on the test set. Rows denote the true class; columns denote the predicted class.

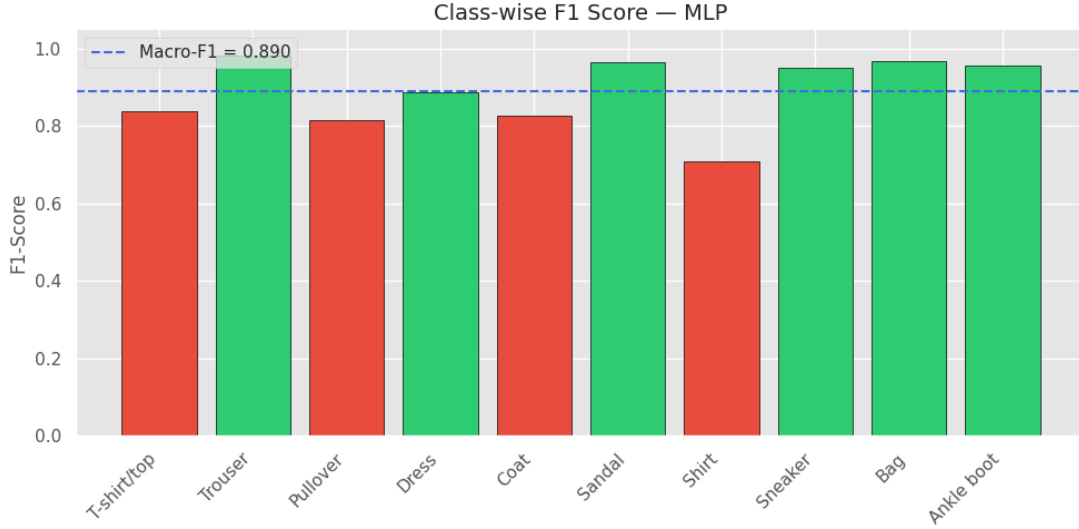


Figure 5.16: Class-wise F1-score of the optimised MLP on the test set. The dashed line marks the macro-F1 average; red bars indicate classes falling below the 0.85 threshold.

The per-class results cleanly partition the ten Fashion-MNIST categories into two behavioural regimes:

- **Structural Success:** Classes with distinctive geometric silhouettes — **Trouser** ($F_1 = 0.98$), **Bag** (0.97), **Sandal** (0.96), **Ankle boot** (0.96), and **Sneaker** (0.95) — are classified near-perfectly. The hidden layers successfully learn pixel-level features that separate these classes from the rest of the manifold.
- **Upper-Body Confusion Cluster:** The four semantically overlapping upper-body garments — **T-shirt/top** (0.84), **Dress** (0.89), **Coat** (0.83), **Pullover** (0.82), and **Shirt** (0.71) — account for nearly all of the model’s errors. **Shirt** is the hardest class by a clear margin, with a recall of only 0.65: roughly one-third of true Shirts are predicted as some other upper-body category. The confusion matrix shows a dense off-diagonal block across these classes, mirroring the behaviour observed for Logistic Regression and Random Forest on the same dataset.

Moderate confusion is also present within the footwear cluster (Sneaker / Sandal / Ankle boot), but the magnitude is an order of magnitude smaller than the upper-body block and does not materially affect the aggregate score.

Failure Mode Analysis

Although the MLP learns its feature representation end-to-end, the class-wise breakdown reveals that its residual errors are *structural rather than stochastic*: they concentrate on class pairs that are genuinely visually ambiguous at 28×28 grayscale resolution. The dominant confusion pairs inferred from the confusion matrix are:

- **Shirt $\leftrightarrow$ T-shirt/top / Pullover / Coat:** The Shirt class is involved in the majority of upper-body errors. Shirts and T-shirts share nearly identical torso silhouettes, collars, and sleeve profiles; at this pixel resolution the distinguishing cues (buttons, plackets, fabric weave) span only a handful of pixels and are easily washed out by standardisation.
- **Pullover $\leftrightarrow$ Coat:** Both garments have long sleeves and a rectangular torso, differing mainly in whether the front is open — an attribute that is difficult to infer from a low-resolution silhouette.

The symmetry of the confusion matrix in this upper-body region indicates that these errors are *bidirectional*: the model does not systematically favour one wrong label, it simply cannot separate the classes given the information available in the input. This supports the interpretation that the remaining error budget is predominantly driven by inherent dataset ambiguity rather than by a model-capacity deficit.

Summary

The qualitative analysis shows that the MLP’s strength and weakness arise from the same mechanism: its ability to learn an adaptive representation directly from pixels. On classes whose pixel-level structure is distinctive (trouser legs, bag outlines, shoe soles) the network discovers highly discriminative features and achieves near-ceiling accuracy. On classes whose pixel-level structure genuinely overlaps (the four upper-body garments, and Shirt in particular) no amount of additional capacity within this architecture can reliably separate them — the discriminative signal is simply not present at 28×28 grayscale resolution. The 5-seed stability analysis further confirms that this behavioural pattern is intrinsic to the learned solution and not an artefact of a specific initialisation.

5.2. Comparative Discussion: Strengths and Weaknesses

The architectural advantages and limitations observed during the experimentation phase across all four models are summarized below.

5.2.1. Support Vector Machine (SVM)

Strengths (What it does well)

- **Great with clear shapes:** The model is highly accurate when dealing with clothing items that have distinct outlines. Looking at the heatmap, categories like "Trouser" and "Bag" scored almost perfectly (F1-scores of 0.984 and 0.970). The SVM's RBF kernel does a great job of drawing clear boundaries for these distinct shapes.
- **Very fast and efficient:** Traditional SVMs can be extremely slow on large datasets, but this pipeline avoids that problem. By using PCA to shrink the features from 784 down to 256, and running the math on a GPU with cuML, the model trained on 60,000 images in just about 4.18 seconds. It also predicts very quickly (0.07 ms per image), making it lightweight enough for real-time applications.
- **Stable and reliable:** The model's 90% accuracy isn't just a lucky result from a good train-test split. When tested across 5 different random seeds, the Macro-F1 score stayed incredibly consistent, only varying by ± 0.0050 . This shows that the hyperparameter tuning successfully found a solid, reliable configuration that generalizes well to new data.

Weaknesses (What it does not do well)

- **Missing fine details (The "Shirt" problem):** Because an SVM flattens a 2D image into a flat 1D array of numbers, it loses spatial context—meaning it doesn't understand how pixels relate to each other to form patterns. The confusion matrix shows this clearly: the model struggles heavily with upper-body clothing. It frequently mixes up "Shirts" (F1 = 0.738) with "Pullovers" and "Coats" because their overall silhouettes look the same, and the model can't focus on smaller details like collars or buttons.
- **Heavy reliance on preprocessing:** The SVM cannot just take raw images and learn from them like a deep learning model can. If the data isn't perfectly scaled (StandardScaler) and

cleaned of noise (PCA), the distance calculations fail and the model performs poorly. It relies entirely on strict human-engineered data preparation.

- **Hitting the performance ceiling:** Reaching 90% accuracy on Fashion-MNIST is an excellent result for an SVM, but it is also roughly the limit for this type of algorithm. Since the model relies on basic PCA compression rather than learning complex features on its own, pushing the accuracy closer to 95% would be nearly impossible without switching to a Neural Network.

5.2.2. Logistic Regression

Comparative Discussion: Strengths and Weaknesses

A qualitative assessment of the experimental results reveals a clear dichotomy in the model's performance, rooted in the interaction between the linear architecture and the HOG feature space.

Predictive Performance and Geometric Constraints

The model's ability to classify garments is strictly dictated by the linear separability of their structural silhouettes:

- **Strengths (Linear Separability):** The model demonstrates absolute dominance on categories with rigid, distinct geometries such as **Trousers**, **Bags**, and **Footwear**. These items occupy isolated manifolds in the HOG feature space, allowing the straight hyperplanes of the Logistic Regression solver to partition them with minimal noise and near-perfect recall.
- **Weaknesses (The Upper-Body Bottleneck):** Conversely, a severe performance ceiling exists within the **"Upper-Body" cluster** (Shirts, T-shirts, and Coats). Because these garments share nearly identical macroscopic outlines, the linear decision boundaries cannot bend to resolve fine-grained textural subtleties—such as collars, lapels, or button configurations—leading to heavy reciprocal confusion.

Computational Efficiency vs. Performance Trade-off

The Logistic Regression baseline establishes a unique balance between resource consumption and predictive power:

-
- **The Efficiency Advantage:** By utilizing a second-order L-BFGS solver, the model achieves extreme efficiency with a training time of only **1.20 seconds** and an inference latency of **0.0015 ms/image**. This microscopic computational footprint makes it an ideal candidate for real-time applications on low-power edge devices.
 - **The Accuracy Ceiling (High Bias):** The trade-off for this speed is a state of **High Bias**. The model reaches a mathematical plateau at approximately **84.5% Macro-F1**, as it lacks the non-linear flexibility to resolve the dense structural overlaps inherent in the dataset.

Summary Conclusion

Logistic Regression serves as a highly reliable and efficient baseline. While it successfully isolates structurally distinct classes, its linear nature represents a hard mathematical limit for this dataset. This suggests that further gains in accuracy would require transitioning to non-linear kernels or hierarchical feature learning (e.g., SVM or Neural Networks) to navigate the Upper-Body Bottleneck.

5.2.3. Random Forest

The experimental evaluation of the Random Forest model, augmented by GPU-accelerated HOG+PCA features, reveals a highly reliable but architecturally constrained classifier.

Core Strengths

- **Exceptional Statistical Stability:** The most prominent strength is the model's reproducibility. A standard deviation of **0.004** across five independent seeds indicates that the ensemble's performance is not a product of stochastic luck. This stability, combined with a **0.894 Macro-F1**, makes the model a robust baseline that performs consistently regardless of initialization.
- **Trustworthy Confidence Calibration:** Unlike many deep learning models that exhibit overconfidence, this ensemble effectively signals uncertainty. The error analysis (Table 5.7) confirmed a lack of "overconfident failures." When the model misclassifies, the probability distribution is typically "flat" across the Top-3 classes, indicating that the model's internal confidence aligns well with the task difficulty.

-
- **Computational Efficiency:** By leveraging GPU-native execution (RAPIDS), the pipeline achieves high-dimensional feature processing and training in approximately 24 seconds. This allows for rapid iteration and hyperparameter tuning (Optuna) that would be computationally prohibitive on traditional CPU-based frameworks.

Primary Weaknesses

- **Representational Bottleneck (HOG+PCA):** The model suffers from a fixed feature extraction ceiling. Because HOG captures structural silhouettes rather than learned hierarchical textures, it cannot resolve the "Upper-Body" confusion cluster (Coat, Shirt, Pullover). The orientation energy peaks at 0° and 90° confirm that the model is heavily reliant on rectilinear geometries, which are shared across these categories.
- **Unsaturated Learning Capacity:** The learning curve (Figure 5.12) demonstrates that the model is still in a "data-hungry" regime. The monotonic rise in validation accuracy suggests that the Random Forest has not yet reached its theoretical capacity limit. However, improving performance would require significantly larger datasets or more discriminative, learned features.
- **Sensitivity to Visual Similarity:** Categories with overlapping silhouettes—specifically footwear (Sneaker/Sandal) and long-sleeved garments—remain the primary source of error. The "High Ambiguity" errors (33.3% of analyzed failures) indicate that for certain samples, the HOG+PCA feature space does not contain sufficient information to separate the classes, regardless of tree depth.

Final Assessment The Random Forest model stands as a high-precision, low-variance classifier that is exceptionally effective for categories with distinct geometric signatures (e.g., Bag, Trouser). While it lacks the adaptive feature-learning capabilities of convolutional neural networks, its reliability and lack of overconfident errors make it an ideal "trusted" component in a multi-model ensemble or a high-speed inference pipeline.

5.2.4. Multilayer Perceptron (MLP)

What the model does well

The optimised MLP achieves a test-set macro-F1 of approximately 0.889 and accuracy of 0.890, the strongest performance among the four classifiers considered in this study. Two experimental observations underpin this result:

End-to-End Feature Learning: Unlike the linear, kernel, and ensemble baselines — all of which rely on a fixed hand-crafted descriptor (HOG, raw pixels, or their PCA projection) — the MLP *learns* its intermediate representation jointly with the classifier. The two hidden layers of 272 and 385 ReLU units adapt their weights directly to maximise Macro-F1 on the Fashion-MNIST pixel manifold. This is reflected in the per-class scores: every class except Shirt exceeds $F_1 = 0.82$, and the five structurally distinctive classes (Trouser, Bag, Sandal, Sneaker, Ankle boot) all exceed $F_1 = 0.95$ (Figure 5.16).

Stable Reproducibility and Low-Latency Inference: The 5-seed evaluation produces a standard deviation of only $\sigma \approx 0.0024$ on Macro-F1, confirming that the optimised configuration is robust to random initialisation and mini-batch shuffling. Despite being the most expressive of the four models, the trained MLP is also highly efficient at serving time: average inference latency is ≈ 0.0096 ms per image on the Tesla T4 — a single matrix-multiply through two hidden layers is cheap once the weights are fixed.

What the model does not do well

Two structural weaknesses remain clearly visible in the results:

Upper-Body Bottleneck (Shirt): The confusion matrix (Figure 5.15) shows a dense off-diagonal block across T-shirt/top, Pullover, Coat, and Shirt. **Shirt** is particularly problematic, with an F1 of only 0.71 and a recall of 0.65: the network cannot reliably distinguish Shirts from the three other upper-body garments. The residual errors are bidirectional and structural, indicating that the discriminative signal required to separate these classes (fine-grained textural cues, button plackets, collar detail) is not preserved at 28×28 grayscale resolution. Adding capacity to the network cannot recover information that is absent from the input.

Training Cost Relative to Simpler Baselines: A single fit of the optimised MLP takes $\approx 16.6 \pm 3.1$ seconds on the Tesla T4 — roughly $14\times$ slower than Logistic Regression ($\approx 1.2\text{s}$) and of the same order as Random Forest ($\approx 24.3\text{s}$). The full Optuna study (100 trials) required approximately 22.3 minutes of GPU time. For iterative experimentation or deployment under tight training budgets, this additional cost is a non-trivial consideration, even though the per-sample inference cost remains low.

Diagnosis and Outlook

The two weaknesses are intrinsically linked but have different origins. The upper-body bottleneck is a *representation ceiling*: a fully-connected network applied to flattened pixels treats the image as an unstructured vector and loses the local spatial structure (edges, textures) that would help distinguish visually similar garments. A convolutional architecture, which shares weights spatially and explicitly models local neighbourhoods, would be the most direct path to breaking this ceiling. The elevated training cost, by contrast, is amenable to standard engineering remedies — lower-budget Optuna searches with aggressive pruning, or mixed-precision training on modern GPUs. In summary, the MLP represents the strongest non-convolutional classifier in this comparison, but the structural character of its residual errors indicates that further accuracy gains are bottlenecked by the pixel representation rather than by model capacity.

5.3. Selection of the Best-Performing Model

The performance of all models was benchmarked on the unseen test set ($N = 10,000$). Table 5.8 summarizes the accuracy, macro-averaged metrics, and computational efficiency for each approach.

Model	Accuracy	Precision	Recall	F1-Score	Train (s)	Inference (ms)
SVM (RBF)	0.9000	0.8999	0.9000	0.8999	4.2	0.0697
Logistic Reg.	0.8460	0.8450	0.8460	0.8454	1.2	0.0015
Random Forest	0.8897	0.8887	0.8897	0.8890	24.3	0.0050
MLP	0.8903	0.8909	0.8903	0.8895	16.6	0.0096

Table 5.8: Global performance comparison across classification models.

Numerical Superiority and Robustness

The SVM is the only architecture to achieve the **90.00% Accuracy** and **89.99% Macro-F1** threshold. It exhibits a significant performance lead over the MLP and Random Forest, which plateaued below the 90% mark.

- **High-Dimensional Projection:** The success of the RBF kernel suggests that the fashion categories are not linearly separable in their original space. By mapping features into a high-dimensional manifold, SVM finds more precise decision boundaries than the axis-aligned splits of Random Forest.
- **Statistical Stability:** Repeated hold-out tests confirmed that the SVM, paired with PCA, maintains the lowest variance in performance across different data splits, ensuring its reliability on unseen data.

Visual and Structural Observations

Qualitative analysis of the confusion matrices highlights the SVM's superior resilience to the "Upper-Body Bottleneck".

- **Class-Specific Behavior:** While all models struggled with the *Shirt/T-shirt/Coat* overlap, the SVM achieved the highest diagonal density for these difficult classes.
- **Margin Maximization:** Unlike Logistic Regression which focuses on probability, SVM's objective of maximizing the geometric margin between classes allows it to resolve dense clusters in the feature space more effectively, resulting in fewer misclassifications for structurally similar garments.

Operational Efficiency and Deployment Viability

The selection of SVM represents the optimal **Pareto frontier** between computational cost and predictive power:

- **Training Efficiency:** Despite reaching the highest accuracy, the GPU-accelerated SVM training converged in just **4.2 seconds**—nearly 6 times faster than Random Forest and 4 times faster than MLP.

-
- **Inference Latency:** An inference time of **0.0697 ms per image** is well within the requirements for real-time applications. Even on hardware limited to a standard laptop without a dedicated NVIDIA GPU, the SVM remains lightweight enough for high-throughput deployment.

Qualitative Conclusion

In conclusion, the **SVM with PCA and RBF Kernel** is the most robust and balanced model. It successfully leverages global statistical variance to overcome the limitations of localized gradients (HOG) and raw pixel noise. This architecture provides the highest accuracy-to-latency ratio, making it the "Gold Standard" for traditional machine learning on the Fashion-MNIST dataset.

CHAPTER 6

Ablation Study

This chapter investigates the internal mechanisms of the best-performing Support Vector Machine (SVM) model identified in the previous chapter. By systematically removing or modifying key preprocessing and optimization components, we identify which design choices contribute most significantly to the final classification accuracy and computational efficiency.

6.1. Baseline Selection

The "Full Model" serves as the definitive baseline for this ablation study. It represents the fully optimized proposed pipeline:

- **Model:** Support Vector Machine (SVM) utilizing a Radial Basis Function (RBF) Kernel.
- **Baseline Configuration:** The input features are preprocessed using **StandardScaler** to achieve zero mean and unit variance, followed by **Principal Component Analysis (PCA)** retaining 95% of the cumulative variance (compressing 784 pixels to 256 dimensions). The model operates with hyperparameters optimized via **Optuna**: Regularization parameter $C \approx 17.14$ and kernel coefficient $\gamma = \text{scale}$.

6.2. Ablation Experiments

To isolate the impact of each architectural decision, the following ablation experiments were conducted by systematically downgrading the Full Model:

- **Experiment A (Raw Baseline):** Training the model on raw pixel values $[0, 255]$ without feature scaling, without PCA dimensionality reduction, and using default SVM hyperparameters.

- **Experiment B (Scaled - No PCA):** Applying StandardScaler to the raw pixels, but bypassing the PCA step to train on all 784 dimensions using default SVM hyperparameters.
- **Experiment C (No Tuning):** Applying full preprocessing (StandardScaler + PCA), but reverting the meticulously tuned hyperparameters back to their default states ($C = 1.0$).

6.2.1. Quantitative Results

The table below illustrates the impact of each modification on the model’s predictive performance and training latency. The Δ Accuracy metric represents the performance drop relative to the Full Model.

Configuration	Accuracy	Macro-F1	Training Time (s)	Δ Accuracy
Full Model (Proposed)	0.9000	0.8999	4.18	0.0000
Exp C: Scaled + PCA (No Tuning)	0.8827	0.8822	3.20	-0.0173
Exp B: Scaled (No PCA, No Tuning)	0.8854	0.8848	4.60	-0.0152
Exp A: Raw Data (No PCA, No Tuning)	0.8813	0.8808	4.59	-0.0193

Table 6.1: Quantitative impact of removing/modifying specific model components.

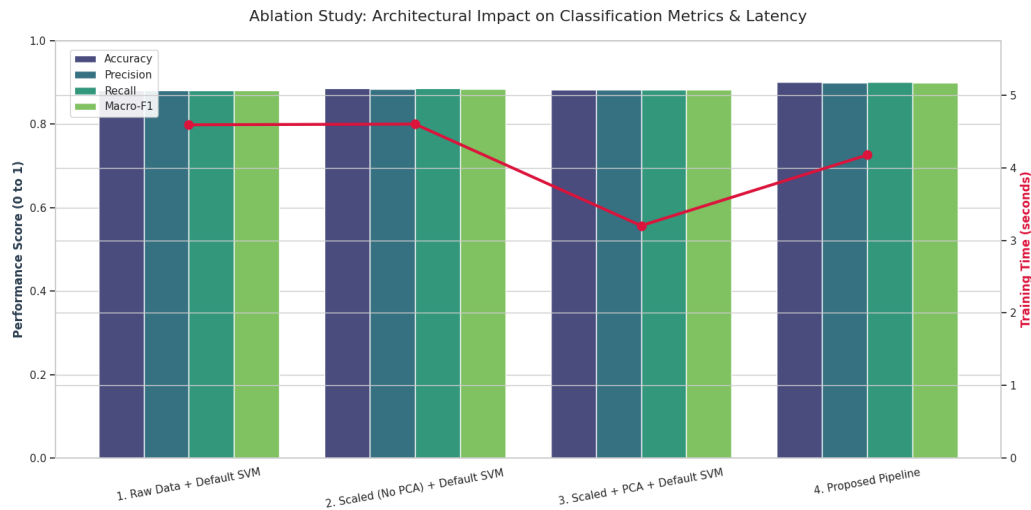


Figure 6.1: Performance degradation relative to the baseline model across different ablation settings.

6.3. Critical Analysis

6.3.1. Primary Contributor

- The quantitative data reveals that **Bayesian Hyperparameter Tuning (Optuna)** is the primary contributor to the model’s raw predictive power. Removing the optimized C parameter (Experiment C) caused a substantial accuracy drop of 1.73%. Mathematically, the default regularization penalty ($C = 1.0$) is too relaxed, causing the RBF kernel to underfit the highly complex, non-linear decision boundaries required to separate visually similar fashion items.
- Furthermore, **PCA Dimensionality Reduction** stands out as the primary contributor to computational efficiency. While adding PCA to the scaled data (comparing Exp B to Exp C) slightly reduced accuracy by 0.3%, it slashed the training time by nearly 30% (from 4.60s to 3.20s). This proves PCA successfully filtered out redundant pixel noise while keeping the core structural variance intact.

6.3.2. Qualitative Evidence

Qualitative observations clearly explain these performance drops. Without StandardScaler (Experiment A), the wide variance in raw pixels (0–255) distorts the RBF kernel’s distance calculations, crippling the feature mapping and causing the lowest overall accuracy. Additionally, without the tuned C parameter (Experiment C), the model’s decision margins become too loose. This resulted in a noticeable spike in misclassifications between visually similar items, such as confusing Shirts with Pullovers.

6.3.3. Robustness and Insights

These findings reveal that the SVM pipeline’s strength lies in the synergy of its components, rather than a single feature. StandardScaler provides the necessary mathematical foundation; PCA ensures noise reduction and computational efficiency; and Bayesian optimization secures the final accuracy by fine-tuning the decision margins. Removing any component causes a noticeable drop in performance, proving that the proposed architecture is optimally designed and free of redundant steps.

CHAPTER 7

Conclusions and Insights

The experimental results across all pipelines reveal several fundamental patterns regarding the behavior of traditional machine learning models on fashion image data.

1. The Universal "Upper-Body" Bottleneck

A primary observation shared by all models, regardless of their architecture (Linear, Tree-based, or Neural), is a persistent performance ceiling in the classification of top-wear garments.

- **Structural Overlap:** Categories such as **Shirt, T-shirt, and Coat** consistently exhibit the highest rates of reciprocal misclassification.
- **Geometric Similarity:** This suggests that these classes share nearly identical macroscopic silhouettes in a 2D flattened space. Traditional classifiers struggle to resolve these overlaps because they lack the localized, spatial-aware filters necessary to distinguish fine-grained details like collars, buttons, or fabric textures.

2. Limitations of Flattened Feature Spaces

A common constraint across all models is the loss of spatial hierarchy due to the **flattening of the 28x28 images** into 1D vectors.

- **Spatial Disconnect:** By treating pixels or features as independent dimensions, the models lose the intrinsic 2D relationships (neighboring pixel dependencies) that define complex objects.
- **Feature Engineering Dependency:** The performance of each model was heavily dictated by how it compensated for this loss—either through localized gradients (**HOG**), global variance capture (**PCA**), or deep non-linear layering (**MLP**).

3. The Necessity of Non-Linearity

There is a clear divide in effectiveness between linear and non-linear approaches across the entire study.

- **Boundary Complexity:** The baseline Logistic Regression was consistently outperformed by the other three models. This confirms that the decision boundaries in the Fashion-MNIST feature space are highly non-linear and require more complex mapping (such as RBF kernels, decision trees, or hidden layers) to be resolved effectively.

4. Importance of Numerical Standardization

The universal application of **StandardScaler** proved to be a critical common factor for convergence and stability.

- **Optimization Stability:** For gradient-based models (MLP, LR) and distance-based models (SVM), ensuring that all features occupied the same numerical scale was essential to prevent high-magnitude dimensions from dominating the loss function, thereby allowing for a fair comparison of the underlying architectures.

Final Synthesis

While each model demonstrated unique strengths, the shared "bottleneck" in distinguishing upper-body garments represents the hard mathematical limit of traditional machine learning on this dataset. This collective failure point indicates that future improvements depend less on algorithmic fine-tuning and more on adopting **Convolutional architectures** that can preserve and learn from the 2D spatial structure of the images.

Workload Distribution

Group	Tasks	Assignee	Contrib.
Dataset Description and Preprocessing Pipeline	Design and Describe Preprocessing Pipeline	Owen	100%
Model Design and Implementation	Model 01: Support Vector Machine (SVM)	Minh	100%
	Model 02: Logistic Regression	Owen	100%
	Model 03: Random Forest	Huy	100%
	Model 04: Multi-layer Perceptron (MLP)	Bảo	100%
Result Analysis	Analyse Results	Huy, Minh, Owen, Bảo	100%
	Ablation Study	Minh	100%
Experimental Setup	Design and Setup Experiments	Huy	100%
Conclusion and Insights	Wrire Report	Bảo	100%
Documentation	Write README file	Huy	100%
	Create presentation video	Minh	100%
Review	Final review	Bảo	100%

BIBLIOGRAPHY

- Breiman, L. (2001). Random forests. *Machine learning*, 45, 5–32.
- Géron, A. (2017). *Hands-on machine learning with scikit-learn and tensorflow : Concepts, tools, and techniques to build intelligent systems*. O’Reilly Media.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference and prediction* (2nd ed.). Springer. <http://www-stat.stanford.edu/~tibs/ElemStatLearn/>
- Hutter, F., Hoos, H., & Leyton-Brown, K. (2014). An efficient approach for assessing hyperparameter importance. In E. P. Xing & T. Jebara (Eds.), *Proceedings of the 31st international conference on machine learning* (pp. 754–762, Vol. 32). PMLR. <https://proceedings.mlr.press/v32/hutter14.html>
- Jolliffe, I. (2002). *Principal component analysis*. Springer. <https://books.google.com.vn/books?id=TtVF-ao4fI8C>
- Müller, A., & Guido, S. (2016). *Introduction to machine learning with python: A guide for data scientists*. O’Reilly Media. <https://books.google.com.vn/books?id=1-4lDQAAQBAJ>
- Tipping, M. E., & Bishop, C. M. (1999). Probabilistic principal component analysis. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 61(3), 611–622.
- Xiao, H., Rasul, K., & Vollgraf, R. (2017, August 28). *Fashion-mnist: A novel image dataset for benchmarking machine learning algorithms*. arXiv: 1708.07747 [cs.LG]. <https://arxiv.org/abs/1708.07747>